

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PULCHOWK CAMPUS

A

PROJECT REPORT

ON

PYTHON CODING ASSISTANT

SUBMITTED BY:

AASHISH SHRESTHA (PUL077BCT003)

DIVYA SHANGKAT KARKI (PUL077BCT028)

KHAGENDRA KARKI (PUL077BCT036)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

April, 2025

TRIBHUVAN UNIVERSIY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"A Python Coding Assistant"** submitted by **Aashish Shrestha**, **Divya Shangkat Karki**, **Khagendra Karki** in partial fulfillment of the requirements for the Bachelor's degree in Electronics & Computer Engineering.

.....
Supervisor
Prakash Chandra Prasad
Assistant Professor
Department of Electronics and Computer
Engineering,
Pulchowk Campus, IOE, TU.

Date of approval:

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
Pulchowk Campus, Institute of Engineering, TU
Lalitpur, Nepal.

Acknowledgements

We are profoundly grateful to the **Department of Electronics and Computer Engineering** for granting us the invaluable opportunity and platform to undertake this project. Their continuous support and encouragement have been instrumental in our learning and development.

Our sincere appreciation goes to **Assistant Professor Prakash Chandra Prasad**, for his unwavering guidance, insightful suggestions, and meticulous evaluation of our project. His expertise and constructive feedback have greatly contributed to the refinement of our work. Also we are equally grateful to our cluster head, **Assistant Professor Dr. Basanta Joshi** for his invaluable feedback, guiding us through planning to development stages of this project

We are also thankful to our peers and mentors who have offered their encourgaement, constructive critiques, and intellectual engagement.We sincerely acknowledge and appreciate the collective efforts, insights, and support of all those who contributed to this project, whether directly or indirectly.

Abstract

The proliferation of large language models (LLMs) has significantly advanced code generation capabilities. However, their substantial computational demands often restrict real-time, on-device applications for individual developers. To address this limitation, a compact, domain-specific code completion assistant tailored for the Python programming language has been developed. Built upon the Qwen2.5-Coder-0.5B model, the Python Coding Assistant (hereafter referred to as PySage) utilizes a pruned, Python-specific vocabulary and adopts Fill-in-the-Middle (FIM) prompting to enhance context-aware code suggestions. Integration with Visual Studio Code is achieved through a custom C++ backend based on llama.cpp, enabling efficient cross-file context extraction and low-latency inference. The system is optimized for minimal response time, ensuring smooth user experience even on resource-constrained hardware. Experimental evaluations indicate that PySage delivers responsive and accurate code completions, thereby expanding access to AI-assisted development for a wider audience of programmers.

Keywords: *Python Coding Assistant, Large Language Models (LLMs), lightweight models, Transformer architecture,Low-latency AI, Fill-in-the-Middle (FIM), Vocabulary pruning, Real-time inference, edge deployment, Developer Tools*

Contents

PAGE OF APPROVAL ii

COPYRIGHT iii

ACKNOWLEDGEMENTS iv

ABSTRACT v

CONTENT viii

List of Figures x

List of Tables xi

List of Abbreviations xii

1 Introduction 1

1.1 Background 1

1.1.1 Problem Statement 1

1.2 Objectives 2

1.3 Scope of the Project 2

1.4 System Requirements and Implementation 3

1.4.1 System Requirements 3

1.4.2 Implementation Process 4

2 Literature Review 6

2.1 Related Works 6

2.1.1 Static Code Completion Strategies 6

2.1.2 Code Completion with Large Language Models 6

2.1.3 Context Aware Code Completion 6

2.1.4 Tokenizer Optimization for Code 7

2.1.5 Model Efficiency and Lightweight Code Models 7

2.2 Related Theory 7

2.2.1 Transformers 7

2.2.2 Tokenization . . . 11
2.2.3 Quantization . . . 11
2.2.4 Abstract Syntax Tree . . . 11
2.2.5 HTTP-Based Communication . . . 11

3 Methodology **13**
3.1 Feasibility Analysis . . . 13
3.1.1 Technical Feasibility . . . 13
3.1.2 Economic Feasibility . . . 14
3.2 Requirement Analysis . . . 14
3.2.1 Functional Requirements . . . 14
3.2.2 Non-Functional Requirements . . . 15
3.3 Methodological Workflow . . . 15
3.4 Pre-Training Methodology . . . 17
3.4.1 Pretraining Methodology Workflow . . . 17
3.4.2 Data Collection . . . 17
3.4.3 Preprocessing for Pretraining . . . 17
3.4.4 Model Search and Selection . . . 19
3.4.5 Autoregressive Generation . . . 20
3.4.6 Model Architecture . . . 20
3.4.7 Architecture Diagram . . . 20
3.4.8 Training . . . 20
3.5 Finetune Methodology . . . 22
3.5.1 Fine-tuning Methodology Workflow . . . 22
3.5.2 Data Collection . . . 24
3.5.3 Tokenization . . . 26
3.5.4 Training . . . 26
3.6 Modifications for Performance Improvement . . . 27
3.6.1 Optimization for Performance Improvement . . . 28
3.7 Accelerated Inference using `llama.cpp` . . . 28
3.8 Integration with Coding Environment . . . 30
3.8.1 System Overview . . . 30
3.8.2 System Implementation . . . 30
3.9 User Flowchart . . . 33

4 Experimental Setup **34**
4.1 Training . . . 34
4.1.1 Libraries and GPU Used . . . 34
4.1.2 Pre-Training . . . 34
4.1.3 Fine Tuning . . . 36

5 System Design **38**
5.1 Frontend System Diagram . . . 39
5.2 Backend System Diagram . . . 40
5.3 Sequence Diagram . . . 41
5.3.1 Extension Initialization Process . . . 41
5.3.2 Code Completion Workflow . . . 42
5.3.3 Context Tracking Flow . . . 43
5.3.4 AI Chat Workflow . . . 44
5.3.5 HTTP-Connection Process . . . 45

6 Result and Discussion **47**
6.1 Result of vocab Pruning . . . 47
6.2 Pre-Processing Results . . . 47
6.2.1 Training Progress and Insights . . . 48
6.3 VS Code Extension Results . . . 52
6.4 Latency Information . . . 57

7 Conclusion **60**

8 Future Improvements **61**
References . . . 61

List of Figures

1.1 Pylance Extension used for syntax correction 1

2.1 Decoder-Only Transformer Architecture Diagram 8
2.2 Positional Encoding Diagram [1] 9
2.3 Computing attention scores from query and key vectors 9
2.4 Self-Attention Diagram 10
2.5 Feed-Forward Neural Network Diagram[1] 10
2.6 Residual Connection Diagram[1] 11
2.7 Abstract Syntax Tree 12
2.8 Hypertext Transfer Protocol[HTTP] 12

3.1 System Block Diagram 16
3.2 Pretraining Block Diagram 18
3.3 Left: (Performance of model on HumanEvalInfilling [2] task, Right: (Performance to size ration) 19
3.4 Decoder-Only Transformer Architecture[1] 21
3.5 Current File[3] 22
3.6 Cross File context relevant to the working file[3] 22
3.7 Ground truth for current file completion[3] 22
3.8 Fine-tuning Block Diagram 23
3.9 Definition in the imported file `order.py` 25
3.10 The middle part is highlighted. `<MISSING_PART>` denotes the section the model is expected to complete. 25
3.11 Code for which FIM is requested 32
3.12 An example of FIM prompt structure 32
3.13 User Flowchart 33

5.1 System Design 38
5.2 System Design - continue 39
5.3 Frontend System Diagram. 39
5.4 Backend System Diagram 40
5.5 Extension Initialization Process Sequence Diagram 41

5.6 Code Completion Sequence Diagram 42
5.7 Context Tracking Sequence 43
5.8 AI Chat Sequence Diagram 44
5.9 HTTP-Connection Sequence Diagram 45
5.10 LRU Cache Sequence Diagram 46

6.1 One of the Original Dataset Python File 48
6.2 Normalized Dataset of above Python File 48
6.3 Training loss during pretraining 49
6.4 Graph of cosine scheduler with warmup 49
6.5 HumanEval Infilling pass@k scores 51
6.6 Training and validation loss during fine-tuning 51
6.7 Graph of Linear scheduler with warmup 52
6.8 Exact Match accuracy at various steps during fine-tuning 53
6.9 Suggestion 1 53
6.10 Suggestion 2 54
6.11 Suggestion 3 55
6.12 Chat with AI - 1 55
6.13 Chat with AI - 2 56
6.14 Figure showing latency information from backend 58
6.15 Figure showing frontend latency 58

List of Tables

3.1 Model Parameter 20

4.1 Training hyperparameters and their significance 36
4.2 Fine Tuning hyperparameters and their heuristics 37

6.1 Result of Vocab Pruning 47

List of Abbreviations

AI	Artificial Intelligence
LLM	Large Language Model
LORA	Low-Rank Adaptation
RTX	Ray Tracing Texel eXtreme
IDE	Integrated Development Environment
FIM	Fill-In-The-Middle
BERT	Bidirectional Encoder Representations from Transformers
PEFT	Parameter Efficient Fine-Tuning
NTP	Next Token Prediction
PSM	Prefix-Suffix Matching
HTTP	Hypertext Transfer Protocol
GGUF	GPT-Generated Unified Format
GIL	Global Interpreter Lock
NLP	Natural Language Processing
BBPE	Byte-Level Byte Pair Encoding
INT8	8-bit integers
KV	Key-Value
RoPE	Rotary Positional Embedding

1. Introduction

1.1 Background

The rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML) has significantly transformed the software development landscape. One of the most impactful innovations in this field is Large Language Models (LLMs), which have demonstrated remarkable capabilities in Natural Language Processing (NLP). These models have paved the way for AI-powered coding assistants, enabling developers to write code faster, reduce errors, and enhance productivity.

Developers today rely on various VS Code extensions to improve their coding experience. Extensions like Python (by Microsoft) provide syntax highlighting and debugging tools, while Pylance enhances IntelliSense with type hints and static analysis. Similarly, AI-driven tools like GitHub Copilot and Tabnine offer code suggestions and autocompletions based on deep learning models. However, these tools are generalized across multiple languages, which often results in less precise suggestions and contextually irrelevant completions for Python developers.

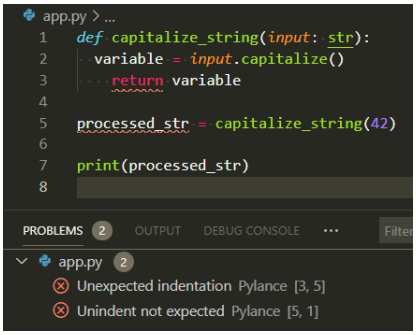


Figure 1.1: Pylance Extension used for syntax correction

Given Python's widespread use in web development, data science, and automation, a Python-specific AI-powered assistant could significantly enhance coding efficiency by providing more accurate, context-aware, and optimized code completions.

1.1.1 Problem Statement

Despite advancements in AI-driven coding assistants, Python developers still face significant challenges when using general-purpose tools, particularly in achieving seamless, real-time

code completion with deep context awareness and minimal latency:

- **Latency Issues:** Large hosted code autocompletion models often face latency issues due to their massive size, remote server deployment, and the need to process long code contexts. Network delays, model loading times, and heavy inference computation can cause noticeable lags, disrupting the real-time experience expected during coding.
- **Lack of Deep Context Awareness:** Enterprise models often lack full project awareness in real-time. The completion suggestion as a result aren't as aware to the context as a user would like them to be. This leads to under utilization of model's true capability.

This project aims to address these challenges by developing a Python-specific AI coding assistant optimized for low-latency and highly context-aware code completion.

1.2 Objectives

The primary objective of this project is to

- Develop a Low Latency Python Code Suggestion Vscode Extension
- Context-Aware Code Completion for Python Programming Language

1.3 Scope of the Project

The scope of this project is to develop a **Python-specific AI-powered coding assistant** as a **VS Code extension**, aimed at enhancing the coding experience for Python developers. Unlike general-purpose AI assistants, this tool will provide context-aware code completions with low latency and optimize suggestions based on Pythonic best practices.

This project covers:

- **Fine-Tuning the Pre-Trained Model:** Utilizing a pretrained transformer-based model, fine-tuned on a large Python dataset to improve code understanding and generation accuracy.
- **VS Code Extension Integration:** Building a seamless interface where the model's suggestions are directly embedded into VS Code, enhancing the developer's workflow.
- **Real-Time AI Assistance:** Providing instant function recommendations and efficient code structuring, reducing manual effort and increasing productivity.
- **Performance Optimization:** Ensuring the model runs efficiently with low latency, making real-time assistance viable for professional and beginner developers alike.

This assistant will be particularly beneficial for students, software engineers, data scientists, and automation engineers, helping them write Python code faster, with fewer errors, and encourage to follow best coding practices.

1.4 System Requirements and Implementation

1.4.1 System Requirements

The development and implementation of the Python coding assistant require efficient computational resources and software tools. The system requirements are classified into **hardware** and **software** requirements.

Since we have developed it as a lightweight Python coding assistant powered by a custom C++ backend built on `llama.cpp`, it still requires a considerable amount of resources to run. The required specification based on our observation of resource usage in the task manager have been reported below.

Minimum Requirements (for basic usage):

- **CPU:** Dual-core processor(Intel i5 / AMD Ryzen 3 or equivalent)
- **RAM:** 8 GB
- **Storage:** 5–10 GB free (for model weights, project context, extension)
- **GPU:** Not required (runs entirely on CPU via `llama.cpp`)
- **OS:** Linux, or macOS
- **Editor:** Visual Studio Code with PySage extension installed

Recommended Requirements (for smooth experience and faster inference):

- **CPU:** Quad-core processor (Intel i7 / AMD Ryzen 5 or better)
- **RAM:** 16 GB or more
- **Storage:** Same storage requirement of 5–10GB is enough
- **GPU:** Optional – not used unless `llama.cpp` is compiled with GPU support (e.g., via cuBLAS or Metal)
- **OS:** Linux or macOS
- **Editor:** Latest version of Visual Studio Code

Dependencies:

- `llama.cpp` compiled with appropriate CPU or GPU flags
- Custom C++ server to handle model inference and communicate with the VSCode extension
- PySage VSCode extension to integrate completions into the editor

1.4.2 Implementation Process

The implementation process is structured into multiple stages to ensure seamless development, training, and deployment.

1. Data Collection and Preprocessing

- We scrapped publicly available repositories

2. Model Fine-Tuning

- Loaded **Qwen/Qwen2.5-Coder-0.5B**[4] from Hugging Face Transformers.
- Fine-tuned using the curated dataset on TensorDock's **RTX 4090 GPU** with mixed-precision training for efficiency.
- Monitored performance using **loss, accuracy, and perplexity** metrics.

3. Model Optimization and Quantization

- We quantized the model to lower precision, reducing the memory footprint and computational requirements during inference.

4. API Development and Integration

- Developed a llama.cpp-based backend with HTTP support for real-time communication.

5. VS Code Extension Development

- Developed the frontend using TypeScript and VS Code API.
- Integrated HTTP-based real-time communication with the API.

6. Testing and Validation

- Conducted unit and integration testing to ensure stable code suggestions.
- Validate the extension result with the testing from friends and project supervisor.

2. Literature Review

Code Completion is a critical feature in modern Integrated Development Environments(IDEs) and coding assistants, enhance developer productivity by suggesting relevant code snippets, reducing syntax errors, and improving efficiency. The development of PySage, a Python code completion assistant stems from the advancement in natural language processing (NLP), deep learning and tokenization strategies for code-related tasks. This literature review related works in this domain and familiarize with the relevant theory.

2.1 Related Works

The development of PySage is informed by numerous advancements in code completion, tokenizer optimization and model efficiency.

2.1.1 Static Code Completion Strategies

Static code completion relies on syntactic and semantic analysis without executing the code. Tools like Microsoft IntelliSense and JetBrains PyCharm leverage abstract syntax trees (ASTs) and type inference to provide context-aware suggestions. Such strategies are efficient and deterministic, offering fast feedback for predictable coding patterns

2.1.2 Code Completion with Large Language Models

(Vaswani et al. 2017[5]) introduced the Transformer architecture, which is the fundamental base for many modern coding assistants. Models like OpenAI's Codex (Chen et al., 2021[2]) and DeepMind's AlphaCode (Li et al., 2022[6]) use self-attention mechanisms to understand and generate complex code completions. Fill-in-the-Middle (FIM) style prompting, as explored by (Bavarian et al., 2023[7]) paves the way for code completion agent leveraging prefix and suffix to generate the middle part.

2.1.3 Context Aware Code Completion

Recent works from (Radford et. al., 2023 [3]) have shown how utilization of cross file context can lead to better suggestion where the model would otherwise fail. They have particularly shown how condensing the cross file information to a single token can lead to better suggestion while optimizing the computational cost by retaining the layer-wise representation of the context token. Sangtani et al., 2024[8] have shown how curriculum based learning for llms can lead to dramatic improvement of llm for code completion. They targeted the AST

nodes for which the model fails to generate correct completion like API call, attribute usage etc. These nodes can be increased in the finetuning process of llms so that they are more familiar with it leading to lower error suggestion.

2.1.4 Tokenizer Optimization for Code

Efficient tokenization is essential for handling programming languages, which have different lexical structures compared to natural language. Sennrich et al. (2016)[9] proposed Byte Pair Encoding (BPE), widely used in models like GPT, while Wang et al. (2022)[10] introduced modifications to BPE specifically for source code. Byte-level tokenizers, such as the one in DeepSeekv2, prevent unnecessary token fragmentation, preserving code integrity, and improving completion accuracy.

2.1.5 Model Efficiency and Lightweight Code Models

Reducing computational complexity without sacrificing performance is a key research focus. Raffel et al. (2020)[11] introduced T5, showing how pre-training objectives affect downstream tasks. Later work by Tay et al. (2022)[11] explored sparse attention mechanisms, lowering memory requirements for long sequences. Techniques like knowledge distillation (Hinton et al., 2015) and vocabulary pruning have proven effective for lightweight, task-specific models.

2.2 Related Theory

The theoretical foundation of the project lies in the recent breakthrough in the NLP domain. With the introduction of attention mechanism[5], Language Models based on this are able to achieve State-of-the-Art performance in various language tasks, surpassing even humans on a few. Here we explore the theories that helped us to develop our assistant.

2.2.1 Transformers

Transformer architectures proposed by Vaswani et. al.[5] have revolutionized natural language processing (NLP) and have been effectively applied to code generation tasks.

Attention Mechanism

This mechanism allows each token to attend to previous tokens in a sequence while generating new tokens. For each word in the sequence, this mechanism helps model decide which other words in the sequence are important and should be "attended to" when processing that word. This is achieved by computing a weighted sum of all words in the input, where the weights (or attention scores) indicate how much importance each word should have in the context of the current word.

Linear transformed is performed on the word to obtain three vectors: query Q, key K, and value V. These vectors are computed as follows:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where W_Q, W_K, W_V are learned weight matrices.

The attention scores are computed using:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k is the dimension of the key vectors.

Decoder-Only Transformers

The decoder-only transformer architecture is a specialized variant of the transformer model, primarily designed for autoregressive tasks where the model generates output sequences based on previously generated tokens. This architecture is particularly effective for applications such as text generation, code autocompletion, and other sequential data processing tasks.

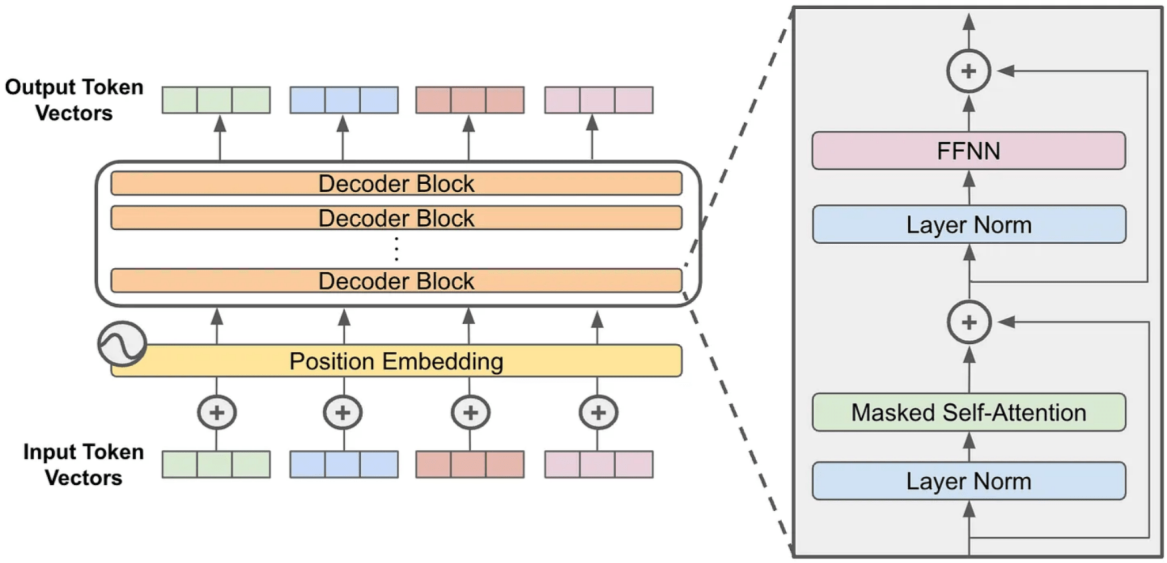


Figure 2.1: Decoder-Only Transformer Architecture Diagram

Key Components

Positional Encoding

Since transformers do not inherently understand token order, positional encodings[12] are added to the input embeddings to provide order-awareness to the model.

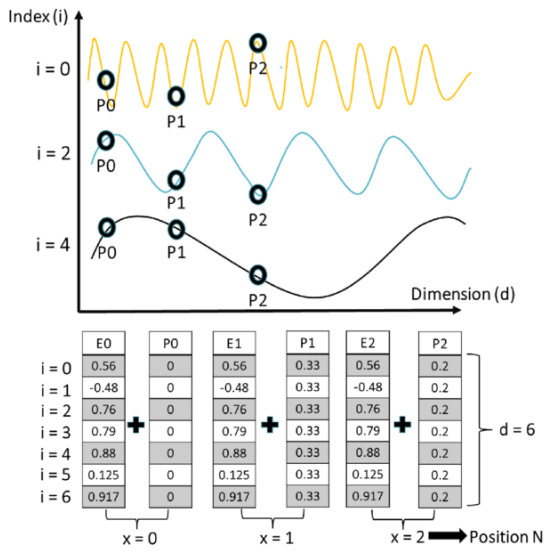


Figure 2.2: Positional Encoding Diagram [1]

Self-Attention Mechanism

Self-attention[5] allows each token to reference all previous tokens in the sequence, enabling the model to understand dependencies and relationships within code snippets.

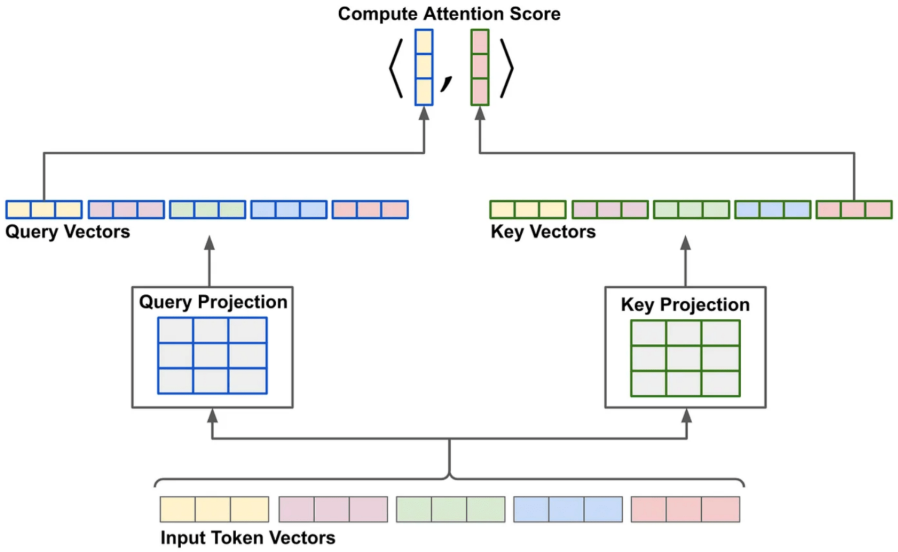


Figure 2.3: Computing attention scores from query and key vectors

Masked Self-Attention

Masked self-attention[13] ensures that the predictions for each token only depend on the preceding tokens, preventing information leakage and maintaining the autoregressive nature

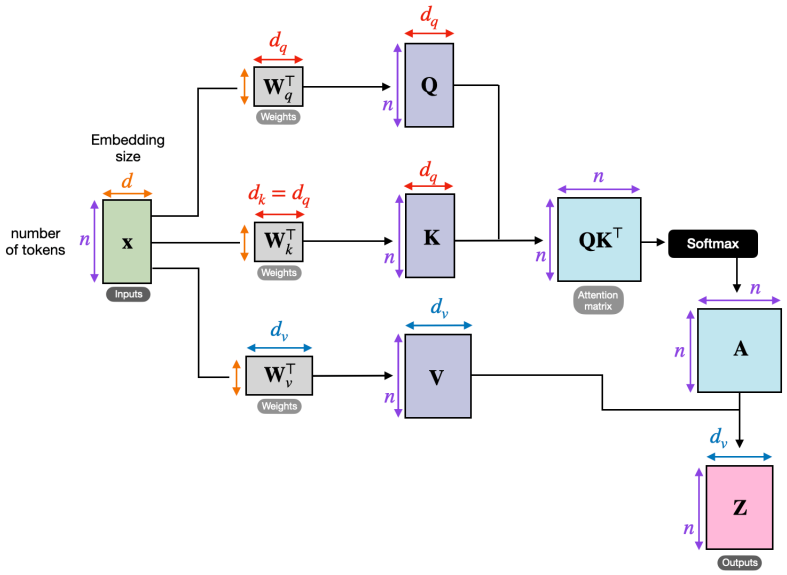


Figure 2.4: Self-Attention Diagram

of the model.

Feedforward Neural Networks

Each token representation is passed through a feedforward neural network (FFN), which enhances feature extraction and transformation.

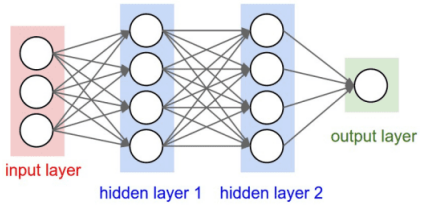


Figure 2.5: Feed-Forward Neural Network Diagram[1]

Layer Normalization and Residual Connections

Layer normalization stabilizes training, while residual connections improve gradient flow and prevent vanishing gradients.

Output Layer

The final layer applies a softmax function over the vocabulary, generating a probability distribution for selecting the next token.

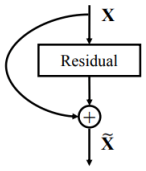


Figure 2.6: Residual Connection Diagram[1]

2.2.2 Tokenization

Tokenization is the process of breaking down text into smaller, meaningful units called tokens. These tokens can be words, subwords, or even individual characters, depending on the approach used. Tokenization is a fundamental step in Natural Language Processing (NLP) because it converts raw text into a structured format that models can understand and process.

2.2.3 Quantization

Quantization is the process of reducing the precision of model weights and activations, typically from floating-point values to lower bit-width representations like 8-bit integers. This reduces the model size and computational requirements, enabling faster inference and lower memory usage, which is crucial for deploying models on resource-constrained devices. Quantization can be done after training (post-training quantization) or during training (quantization-aware training) to minimize accuracy loss and improve efficiency.

2.2.4 Abstract Syntax Tree

Abstract Syntax Tree (AST) is a core data structure in the field of programming language processing, representing the hierarchical and logical structure of source code in a tree format. Unlike concrete syntax trees, ASTs abstract away syntactic details (such as parentheses and commas) and focus on capturing the semantic essence of code constructs. They are extensively employed in compilers, interpreters, static analysis tools, and code transformation frameworks. The AST plays a crucial role in tasks such as syntax checking, code optimization, transformation, and program understanding. Its ability to facilitate high-level reasoning about code structure makes it essential for modern software engineering tools and research (Aho et al., 2006; Baxter et al., 1998).

2.2.5 HTTP-Based Communication

HTTP-Based Communication refers to the exchange of data between clients and servers over the Hypertext Transfer Protocol (HTTP), which forms the foundation of data communication on the World Wide Web. It is a stateless, application-layer protocol designed for distributed,

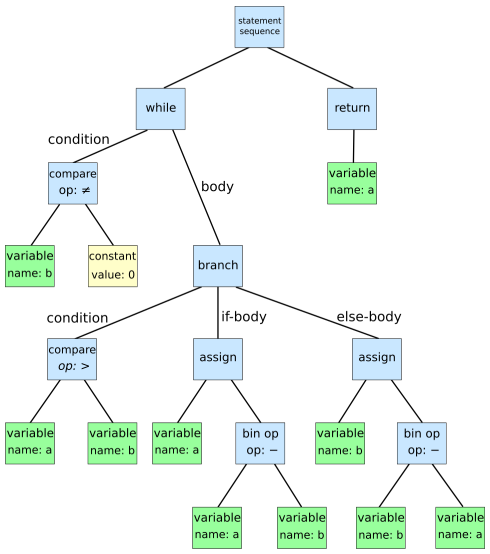


Figure 2.7: Abstract Syntax Tree

collaborative systems. HTTP enables clients (such as browsers or applications) to send requests (e.g., GET, POST, PUT, DELETE) and receive responses, typically in formats like HTML, JSON, or XML. Its simplicity, ubiquity, and support for RESTful architectures have made it a dominant protocol for web APIs and microservice-based systems (Fielding & Taylor, 2000). Modern applications leverage HTTP for interoperability, scalability, and cross-platform communication, often enhanced by secure variants such as HTTPS.

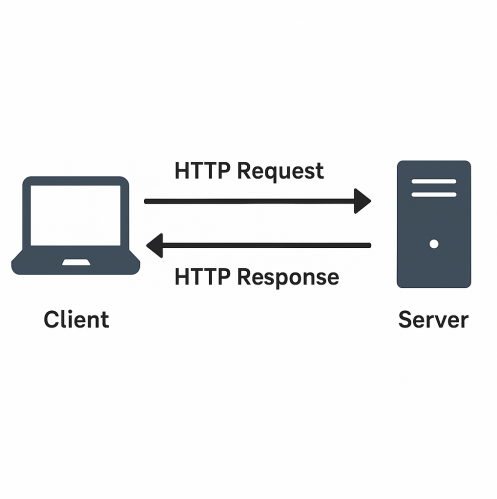


Figure 2.8: Hypertext Transfer Protocol[HTTP]

3. Methodology

In this project, we have adopted a structured methodology to systematically develop Python Coding Assistant Extension. The method has been designed to address critical problems for Python programmers, particularly in coding efficiently and debugging the code. Many programmers spend much time writing code manually that can be generated in seconds, and face problems because of syntax mistakes, contextual inconsistencies, or other internal problems.

To mitigate these challenges, an AI-powered Python coding assistant has been developed on top of a fine-tuned, pretrained decoder-only transformer model. The assistant can provide real-time auto-completion suggestions while developers type, thereby enhancing coding efficiency, accuracy, and speed without sacrificing the user experience. The model has been integrated into Visual Studio Code (VS Code) through a custom extension, enabling developers to access intelligent code assistance directly from their development environment without interrupting the workflow.

The application of a formal framework has facilitated clear problem definition, reproducibility, and methodical improvement. The framework has prescribed data collection, model selection, training protocols, and evaluation and resulted in optimal performance and reliability of the coding assistant

In the pursuit of addressing the challenges faced by Python developers in real-time code generation, this section outlines a comprehensive methodology that encompasses all critical stages of the project life cycle. The approach has been structured to ensure a systematic progression from feasibility analysis to pretraining, fine-tuning, and final deployment as a fully integrated coding assistant within the Visual Studio Code environment.

3.1 Feasibility Analysis

Before proceeding with the development of the Python-specific AI-powered coding assistant, it is crucial to evaluate its feasibility across different dimensions: economic and technical feasibility. This analysis ensures that the project is viable, sustainable, and capable of delivering the intended benefits.

3.1.1 Technical Feasibility

Our coding assistant is technically feasible due to recent advancements in efficient transformer architecture, open-source model availability. By limiting the scope to just the Python

programming language, we can leverage small model that still provides strong performance while remaining lightweight enough for local or edge development. The usage of llama.cpp enables the usage of optimized inference libraries like CuBLAS or ggml-based backends has led to further reduction in latency and memory usage. Further more, vocab pruning, extended pre-training and fine tuning for code-completion specific use case enhanced both the efficiency and completion quality. Overall, the required components - efficient models, python parser and editor integration make our implementation highly practical and manageable.

3.1.2 Economic Feasibility

PySage is designed to be a lightweight and efficient code completion agent aimed specifically for Python developers. Its economic feasibility comes from its low infrastructure demands, targeted functionality, and open-source nature. Unlike large-scale hosted models (e.g., Copilot, CodeWhisperer, Cursor), PySage can run on modest low-end hardware or edge devices, significantly removing cloud inference costs and dependency on third-party platforms. This makes PySage a viable option for coding beginners and learners.

By focusing on Python only and optimizing for fast suggestions, Our system avoids the need for massive models and costly GPU resources. Its modular design also allows selective updates and custom fine-tuning without retraining entire models, further minimizing maintenance expenses.

3.2 Requirement Analysis

The development of *PySage*, a Python Coding Assistant, required a careful assessment of both functional and non-functional requirements to ensure the system meets user needs while maintaining performance, efficiency, and usability. This section outlines the key requirements identified during the planning phase.

3.2.1 Functional Requirements

Functional requirements define the specific behaviors and functionalities that the system must perform. For PySage, the core functionalities include:

- **Real-Time Code Completion:** The system should provide intelligent, context-aware Python code suggestions as the user types in Visual Studio Code.
- **Fill-in-the-Middle (FIM) Support:** The system should be capable of completing code segments based on both prefix and suffix context, enabling FIM prompting.
- **Cross-File Context Extraction:** The system should extract and utilize relevant information (e.g., function definitions, class names, imports) from multiple files in a

project to improve suggestion accuracy.

- **VS Code Integration:** The system should be integrated into the Visual Studio Code editor as an extension that communicates with the backend server via HTTP.
- **Chat-Based Interaction:** The system should allow users to query the model through a built-in chatbox for general coding assistance or explanations.
- **Suggestion Insertion and Acceptance:** The user should be able to accept, reject, or ignore suggestions using predefined keybindings in the editor.

3.2.2 Non-Functional Requirements

Non-functional requirements describe how the system should perform and the constraints under which it must operate:

- **Low Latency:** The system should respond to code completion requests within a second on recommended hardware.
- **Lightweight Deployment:** The system should be deployable on devices with limited resources (e.g., 8–16 GB RAM) without requiring GPU acceleration.
- **Platform Compatibility:** The system should run on Linux and macOS environments with support for future extensions to Windows.
- **Scalability and Modularity:** The architecture should support future model upgrades or component replacements (e.g., tokenizer, model weights) without redesigning the entire system.
- **Security and Privacy:** The system should operate entirely offline with no code data sent to external servers, ensuring user privacy.
- **Ease of Use:** The extension should install with minimal configuration and provide an intuitive interface within the existing development environment.
- **Robustness:** The system should handle incomplete or malformed code gracefully, avoiding crashes or invalid suggestions.

3.3 Methodological Workflow

This figure represent flowchart outlines the end-to-end development process of a Python code suggestion system. It begins with data collection and preprocessing, followed by pretraining

and fine-tuning a model on code completion tasks. The model is then exposed via an HTTP API and further optimized for performance using llama.cpp for accelerated inference. Finally, the model is integrated into a VS Code extension, tested for code suggestions, and brought to completion.

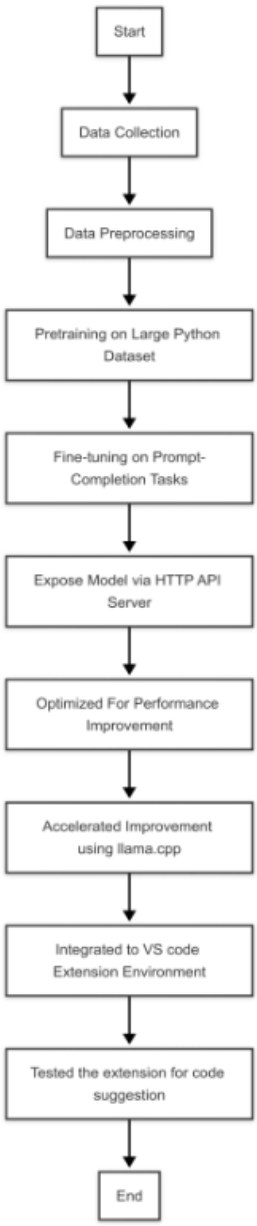


Figure 3.1: System Block Diagram

3.4 Pre-Training Methodology

3.4.1 Pretraining Methodology Workflow

The pretraining phase focuses on teaching the model to understand Python code by exposing it to a large and diverse corpus of Python code from various sources. The goal is to learn general syntax, coding structures, and patterns that are common in Python programming.

The pre-training methodological workflow of the system is illustrated in Figure 3.2. It outlines the key steps from data preparation to model deployment and integration within the VS Code extension.

3.4.2 Data Collection

Initial Corpora Discarded We initially assembled large-scale Python corpora (The Stack[2] and CodeSearchNet[14]), totaling roughly 40 GB raw and refined to 20 GB after cleaning. However, to avoid overlap with the model's original pretraining data and reduce redundancy, these initial datasets were set aside.

Fresh Scraped Repositories : For further pretraining under the pruned-vocabulary regime, we instead scraped **about 4,500 Python repositories** from GitHub between **February 1, 2024** and **February 1, 2025**. The reason we started from February 1 2024 is because the knowledge cutoff of qwen[4] is January, 2024 so any repositories from that date onward will be a new data for the model. Hence, training on this dataset could potential leads to more generality of the model. All the public repositories for which the primary programming language was python were scrapped off.

This yielded approximately 12GB of Python repositories. After further cleaning we were left with 4.5 GB of python files. The cleaning included the following :

1. Removing non-Python files and docs.
2. Filtering out syntax-invalid files.
3. The ratio of numbers to to the total character is greather than 0.6.

3.4.3 Preprocessing for Pretraining

- **Tokenization:** Applied pruned vocabulary with 1024-token max context; sequences truncated with 256-token overlap.
- **FIM Preprocessing:** Introduced Fill-in-the-Middle masks on 50ing pretraining.
- **Data Split:** 95% training, 5% held out for validation.

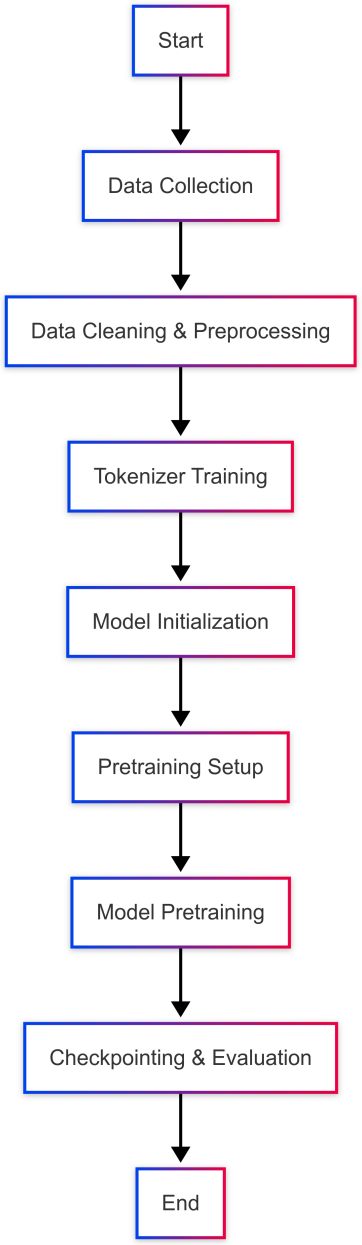


Figure 3.2: Pretraining Block Diagram

3.4.4 Model Search and Selection

Initially, we experimented with multiple models, including **DistilBERT[15], CodeParrot, DeepSeek, and Qwen**. Each was evaluated based on **accuracy, response time, and computational efficiency**. However, some models lacked strong contextual awareness, while others were computationally expensive. We then focused on few specific things like only focusing in small LLMs which can be run locally, trained for FIM and has the latest architectural pattern (judged based on their year of release). After testing quite a bit, we finalized **Qwen/Qwen2.5-Coder-0.5B[4]** as it provided the best balance between **performance, accuracy, and efficiency**.

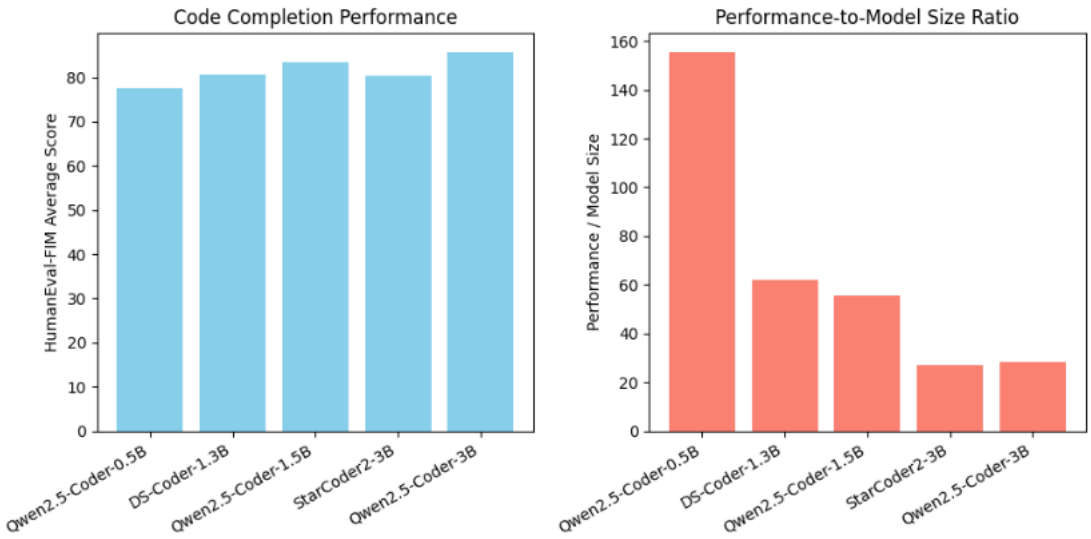


Figure 3.3: Left: (Performance of model on HumanEvalInfilling [2] task, Right: (Performance to size ration)

This model was chosen due to its:

- **Strong Code Understanding:** Pretrained on diverse coding datasets, allowing it to generate accurate and context-aware code completions.
- **Efficient Performance:** With 0.5 billion parameters, it balances computational efficiency and predictive accuracy, making it suitable for real-time coding assistance.
- **Autoregressive Capabilities:** The model generates code token by token based on previously generated content, aligning well with the decoder-only architecture's strengths.
- **Fine-Tuning Potential:** Qwen2.5-Coder-0.5B can be further fine-tuned on domain-specific codebases to enhance performance for specialized applications.

3.4.5 Autoregressive Generation

Autoregressive generation enables the model to predict each token based on prior context, facilitating coherent and contextually relevant code completions. This approach allows the model to dynamically adapt to user input and generate meaningful suggestions.

3.4.6 Model Architecture

The model follows a **decoder-only transformer** architecture, which is inherently autoregressive and excels at sequence generation tasks such as code completion. Table 3.1.

Parameter	**Value**
Hidden Size	896
Layer	24
Query Heads	14
KV Heads	2
Head Size	128
intermediate size	4864
Embedding Table	90869
Parameter	439,332,864
Non Embedding Parameter	357,914,240

Table 3.1: Model Parameter

3.4.7 Architecture Diagram

For a visual representation of the architecture, refer to the Figure 3.4 below:

To fine-tune the pretrained model for code completion tasks using a pruned vocabulary, a carefully designed training regimen was followed. The goal was to enable the model to adapt to the new tokenization scheme while improving its performance in next-token and fill-in-the-middle scenarios.

3.4.8 Training

The model was trained using the **causal language modeling** objective, where it learns to predict the next token in a sequence given all previous tokens. To improve its flexibility in handling real-world code editing scenarios, **Fill-in-the-Middle (FIM)** training was also introduced during preprocessing with a 50% probability. This encouraged the model to generate missing segments when given both prefix and suffix context.

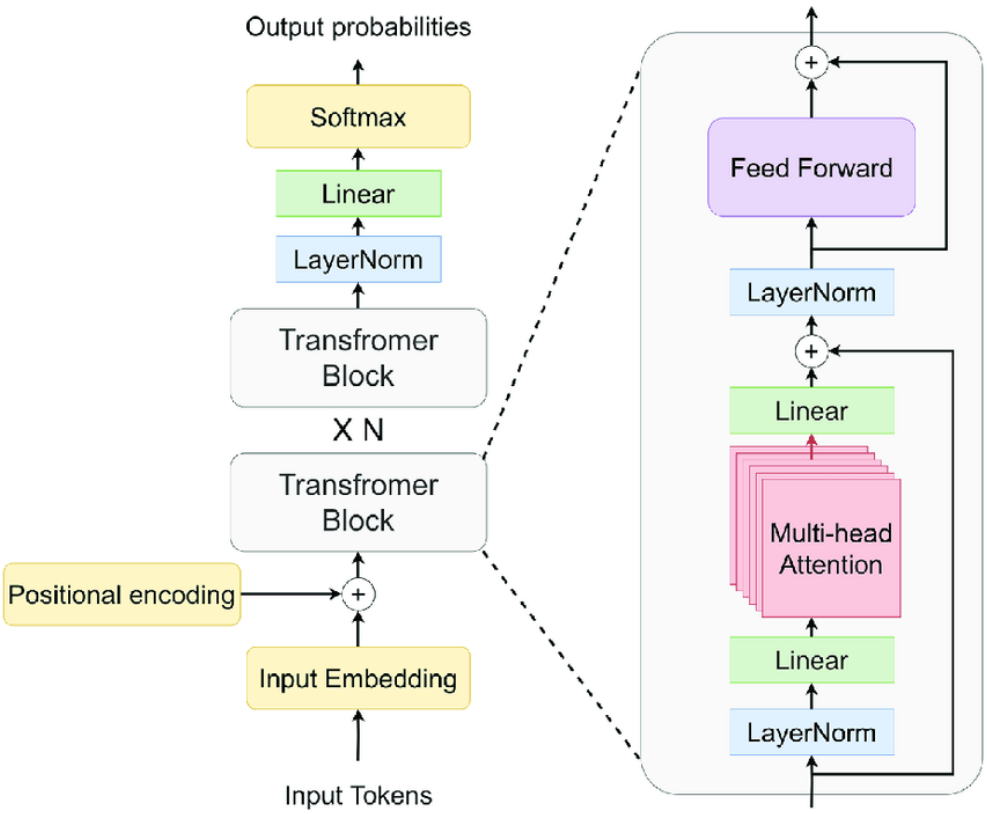


Figure 3.4: Decoder-Only Transformer Architecture[1]

Input Handling and Context Management

- **Context Length:** Sequences were truncated to a maximum of 1024 tokens.
- **Context Overlap:** A 256-token overlap was introduced between consecutive samples to maintain continuity across windows.
- **Tokenization:** Inputs were tokenized using a pruned vocabulary, ensuring adaptation to a reduced but efficient token set.

Model Adaptation to Pruned Vocabulary

A central goal of this training phase was to adapt the model to a **custom pruned vocabulary** optimized for code. This process resulted in:

- **Improved Token Efficiency:** Shorter sequences representing the same code.
- **Faster Inference:** The tokens which were pruned can be removed from the lm_head of the model. Hence those weights can be removed which leads to faster inference.

- **Domain Specialization:** Better modeling of code-specific patterns by removing rarely-used general-purpose tokens.

3.5 Finetune Methodology

Coding often requires working on different files. These files are referenced via imports. This makes the entire codebase more manageable, easier to navigate and isolate debug if any are found. Thus, a context from just from the working file might not always be enough to generate completion. Such case is demonstrated in the figures below. Figure 3.5 is the file which we currently are working on, Figure 3.6 is the cross file context which we are intendnig to reference and figure 3.7 shows our actual completion intention.

Current file to be completed

```
from .tag import TagHandler
from . import git

def main(target, label):
    click.secho(
        'Fetching tags from the upstream'
    )
    handler = ????
```

Figure 3.5: Current File[3]

Cross-file Context

```
# git.py
def list_tags():
    """Returns a list of tags"""
    _fetch_tags()
    return [t for t in _call('git tag')

# tag.py
class TagHandler:
    def __init__(self, raw_tags):
    ...
```

Figure 3.6: Cross File context relevant to the working file[3]

```
TagHandler(git.list_tags())
```

Figure 3.7: Ground truth for current file completion[3]

3.5.1 Fine-tuning Methodology Workflow

The fine-tuning phase focuses on adapting the pretrained model to specific tasks, such as Python code completion, error detection, and debugging. In this phase, the model is exposed to task-specific datasets to learn the nuances of completing code, debugging, and providing relevant suggestions within the context of a VS Code extension.

The fine-tuning methodological workflow of the system is illustrated in Figure 3.8. It outlines the key steps from selecting a fine-tuning dataset to model evaluation and integration within the VS Code extension.

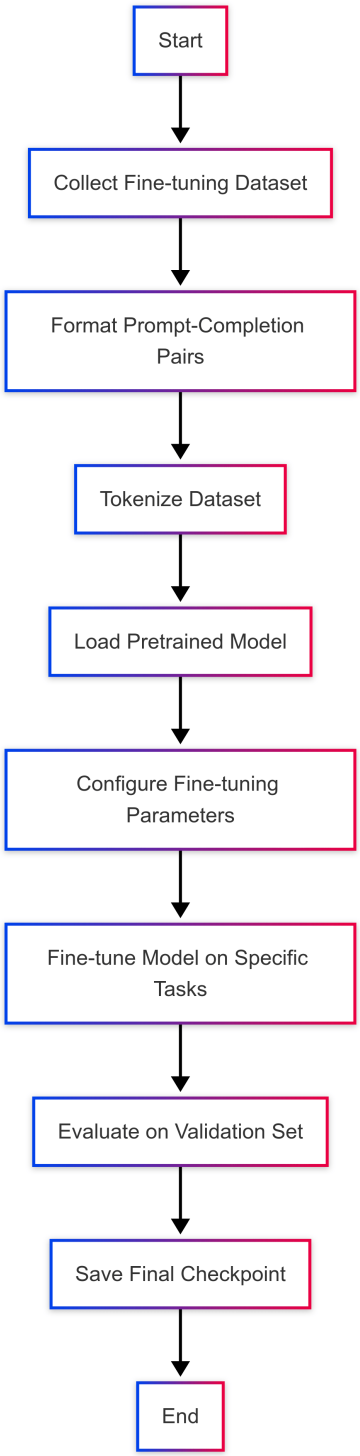


Figure 3.8: Fine-tuning Block Diagram

3.5.2 Data Collection

We curated **about 900** additional Python repositories from GitHub between **February 1, 2025** and **April 3, 2025**. Repos that were made public and for which the primary language was flagged as python were kept. To enable faster download, we implemented a concurrent downloading program to scrape of the repositories. The program would first download clone the repos and keep only the python files. The files were then converted to json data with essential data like file name, file path kept as json properties of the file.

Context Extraction via Tree-sitter

Using Tree-Sitter we first determined the imports in a module. Then we resolve those imports to their path. Then the top-level class definition, function definition and variables are extracted as the possible context. Using Tree-sitter, we parsed each file's AST and located:

- **API calls** (e.g. `obj.method(...)`),
- **Attribute accesses** (`obj.attr`), or
- **Global variables** imported from other modules.

Then those nodes were observed if they used cross-file references in them by matching the identifier of function, or class. If they do then those nodes are extracted and are treated as the middle part of the code completion. The model's task thus is to predict this part based on the cross-file context. To be specific, the completion task included class initialization, class method usage, imported function usage and global variable usage in an expression statement. Lloss were only computed for the middle span. As the context can be very large so we allowed upto 32k tokens. Any data samples existing this limit were excluded.

An examle is showin in figure 3.9 and 3.10. Figure 3.9 consists of the defintion of the class `Order` which is imported in `main.py`. The initialization of the class `Order` thus requires context from `order.py`. Thus we extracted this node out and removed the initialization which the model has to learn to generate.

Using this method we collected about $\sim$ 30,000 data samples. Multiple samples were collected from each repositories.

Example Formatting

```
FILE NAME2
CONTEXT1<|file_sep|>
```

```python
class Order:
    def __init__(self, customer_name):
        self.customer_name = customer_name
        self.items = []

    def add_item(self, name, quantity, price):
        if quantity <= 0 or price < 0:
            raise ValueError("Error")
        self.items.append({
            "name": name,
            "quantity": quantity,
            "price": price
        })

    def remove_item(self, name):
        self.items = [item
                      for item in self.items if item["name"] != name]

    def get_total(self):
        return sum(item["quantity"]
                   * item["price"] for item in self.items)

    def summary(self):
        lines = [f"Order Summary for {self.customer_name}:"]
        for item in self.items:
            lines.append(f" - {item['name']} x{
                item['quantity']} @ ${item['price']:.2f} each")
        lines.append(f"Total: ${self.get_total():.2f}")
        return "\n".join(lines)
```

Figure 3.9: Definition in the imported file `order.py`

```python
# main.py

from order import Order

def run():
    order = Order<MISSING_PART>

    try:
        order.add_item("Laptop", 1, 1200.00)
        order.add_item("Mouse", 2, 25.50)
        order.add_item("Notebook", 5, 3.75)

        print(order.summary())

        print("\nRemoving 'Mouse' from the order...\n")
        order.remove_item("Mouse")

        print(order.summary())

    except ValueError as ve:
        print("Validation error:", ve)
    except Exception as e:
        print("Unexpected error:", e)

if __name__ == "__main__":
    run()
```

Figure 3.10: The middle part is highlighted. `<MISSING_PART>` denotes the section the model is expected to complete.

FILE NAME2
CONTEXT2<|file_sep|>
<|fim_prefix|> <prefix> <|fim_suffix|> <suffix><|fim_middle|><middle><|end_of_text|>

Splits and Sampling

- **Train:** 90% of samples ($\sim$ 27,000).
- **Test**: 5% of samples ($\sim$ 1,500).
- **Validation:** 5% ($\sim$ 1,500), split
- **Tokenization:** pruned tokenizer with 1024 max length, truncation is done whenever if the prompt exceeds the length. Context arround the middle part is kept.

3.5.3 Tokenization

In our project, we used a Byte-Level Byte Pair Encoding (BBPE) tokenizer to process both Python code and English text. To optimize the tokenizer for our use case, we pruned the original vocabulary from **151,646** tokens down to **90,869** tokens, removing unnecessary or rarely used tokens. This helped reduce model complexity and memory requirements, making the tokenizer more efficient for Python-specific code completion and English instructions.

By trimming the vocabulary, we ensured the tokenizer still captured essential code structures, keywords, and common programming patterns while discarding extraneous tokens. This approach not only speeded up tokenization, but also helped streamline model training and inference, making the system more focused and responsive.

3.5.4 Training

After the model was adapted to the pruned token space, it underwent a final fine-tuning stage tailored to realistic code completion scenarios. This stage emphasized learning from examples that mirror real-world usage, such as developer prompts preceded by relevant code context.

Prompt-Context Concatenation

Each training sample was constructed by concatenating a **context block** (e.g., prior code, function definitions) with a **prompt** (e.g., a partial statement, function header, or inline comment). The model was trained to continue the sequence beginning at the prompt boundary.

- Inputs took the form: `<cross file context> + <in file context>`, with a total length of upto 32k.

- The model was supervised to generate only the continuation beyond the prompt.

Loss Masking Strategy

To simulate inference-time behavior, only the tokens that followed the prompt were considered in the loss function. This trained the model to treat the context as fixed and to focus its prediction power on generating meaningful completions.

Fill-in-the-Middle Samples

We followed the FIM prompt structure as defined in the Qwen paper[4], with context from different files concatenated to the main file. A special |file_sep token is used to differentiate the files from one another so that the model knows the content comes from separate sources. The overall prompt structure is:

`FILE NAME`
`CONTEXT<|file_sep|>`
`<|fim_prefix|> <prefix> <|fim_suffix|> <suffix><|fim_middle|><middle>`

These samples trained the model to predict the `<middle>` code segment while treating the prefix and suffix as contextual anchors. FIM samples were interleaved with standard next-token samples to ensure robustness across editing scenarios.

Objective of Fine-Tuning

First objective of finetuning was to train the model to attend to the cross-file context to generate completion. This aligns with the real world scenario where modular development is performed. This also boots the model's ability to condition on large amounts of prior information when generating completions.

3.6 Modifications for Performance Improvement

Several enhancements can be applied to optimize the model's performance:

- **Optimization Strategies:** Implementing mixed precision training and quantization to improve computational efficiency.
- **Memory and Latency Optimization:** Using caching mechanisms to reduce latency.
- **Mixed Precision Training:** Utilizing bf16 data types instead of the traditional fp16 leads to efficient utilization of tensor cores in modern generation GPUs leading to faster training time.

3.6.1 Optimization for Performance Improvement

To enhance model's performance and efficiency, we applied several optimization techniques. These optimizations aimed to reduce memory usage, speed up inference, and streamline the model's capabilities for a more responsive coding assistant experience.

Quantization for Faster Inference

We quantized the model to lower precision, reducing the memory footprint and computational requirements during inference. This step accelerated the response time, making real-time autocompletion more practical.

- Precision Reduction: The model weights were first converted to full precision (FP32) then to 8-bit integers (INT8). This was done using the gguf[16] library. The end format is the gguf format which the llama.cpp natively supports.
- Latency Improvements: Inference speed increased by approximately 1.4, making the coding assistant more responsive.
- Trade-offs: Via manual inspection, we did experience a little degredation in quality. This however is a trade off for the increased inference that quantization support.

Model Pruning for Token Optimization We pruned the model by removing unused tokens, reducing unnecessary computational overhead and trimming the vocabulary to only essential tokens. This pruning step helped streamline the tokenization process and allowed the model to focus solely on Python and English-related tokens.

- **Token Pruning Strategy:** Tokens unrelated to Python syntax or general English language structures were removed.
- **Vocabulary Size Reduction:**The vocabulary was reduced to only relevant tokens, optimizing the model's capacity.
- **Performance Gains:** Reduced memory consumption and faster token decoding without impacting functionality.

3.7 Accelerated Inference using `llama.cpp`

To squeeze out further performance gain, we used `llama.cpp` as the inference engine. This lightweight, C++-based backend is highly optimized for CPU and GPU inference with quantized models, making it ideal for low-latency applications.

Key Features and Benefits:

- **Quantized Execution:** `llama.cpp` supports quantized weights, reducing memory usage while maintaining reasonable accuracy for code generation.
- **Fast Inference via KV Caching:** During inference, `llama.cpp` implements **key-value (KV) caching** which stores intermediate attention results. This allows for efficient reuse of previously computed context, minimizing computations.
- **Prompt Overlap Optimization:** To reduce computation when handling similar prompts, `llama.cpp` computes the longest overlapping prefix between the current and previous prompt. The position embeddings are then shifted appropriately using the Rotary Positional Embedding (RoPE) offset technique, allowing the model to extend or continue prompts without recomputation of unchanged context.

Impact on Performance:

- Improved response times for both short and long prompts due to partial reuse of the previous prompt context.
- Enhanced user experience in real-time coding environments where partial re-prompts are common.
- Lower inference cost on CPU-only environments, making it ideal for lightweight deployment scenarios.

By combining model quantization, token pruning, and optimized inference through `llama.cpp`, the resulting system delivers fast, context-aware completions suitable for real-time developer assistance.

High-Performance C++ Inference Server

To further minimize latency and ensure high-throughput inference, the backend inference server was implemented in C++. Unlike Python-based servers, which often introduce higher overhead due to dynamic typing and interpreter latency, the C++ implementation benefits from:

- **Low-Level Optimization:** Direct memory access and system-level threading provide more control over performance tuning.
- **Efficient Resource Utilization:** C++ enables tighter integration with hardware capabilities (e.g., SIMD, multi-threading), making it more efficient under load.
- **Seamless Integration with `llama.cpp`:** Since `llama.cpp` itself is written in C++, building the server in the same language allows for faster data flow, shared memory buffers, and minimal cross-language I/O overhead.

- **Reduced Latency:** By avoiding Python's Global Interpreter Lock (GIL) and other interpreter-related bottlenecks, the server can serve requests concurrently.

This architecture ensures the model remains highly responsive, even when deployed in production environments or running on edge devices with limited resources.

3.8 Integration with Coding Environment

3.8.1 System Overview

Python Coding Assistant is a VS Code extension that provides AI-driven code autocompletion and chat-based programming assistance. The system consists of a frontend extension integrated into VS Code and a backend AI service that processes user input, generates intelligent code completions, and offers contextualized suggestions.

The backend leverages Tree-sitter for repository-wide context extraction, Prefix-Suffix Matching (PSM) for Fill-in-the-Middle (FIM) task structuring, and Qwen as the AI model for code generation. lama.cpp serves as the inference engine, allowing for fast, efficient model execution. Critical backend components were written in C++ to maximize performance and speed, with HTTPS ensuring secure, real-time communication.

The project began with an analysis of the developer workflow within VS Code, identifying opportunities where AI assistance could significantly improve efficiency. The goal was to create a system capable of deeply understanding the codebase, generating context-aware code, and delivering real-time suggestions without requiring extensive manual prompting.

3.8.2 System Implementation

VS Code Extension

The VS Code extension is developed using TypeScript and Node.js, facilitating seamless interaction between the user and the AI-powered backend. The implementation follows these key steps:

- **Creating the VS Code Extension:** The extension is initialized using the VS Code Extension API, defining commands and event listeners to trigger AI-based code completions and chat interactions.
- **User Interaction Mechanism:**
  - **Chatbox Interaction (Ctrl + Alt + A):** When the user presses Ctrl + Alt + A, the extension opens a chatbox. The chatbox establishes communication with the backend and facilitates real-time interaction with the AI, allowing the user to send queries and receive responses in a conversational format.

  - **Code Autocompletion** Based on the cursor position, the extension gathers the context of the active file (prefix, suffix, etc.) and requests AI-driven autocompletion of the code.

- **Backend Communication:** For **interactions**, the extension uses **HTTPS** to enable bidirectional, real-time communication, allowing it to send user input and receive interactive responses seamlessly.

For **code completion requests**, the extension sends the following parameters to the backend via **HTTPS** to generate context-aware code completions: getFIMCompletion(inputPrefix, inputSuffix, prompt, this.app.extraContext.chunks, nindent)

Backend Server

The IDE sends all the suggestion requests to the backend. The backends first receives the requests parses to json for easy handling, then it extracts the code from the requests. The code is then converted to FIM format as show in the figure3.12. It is then tokenized and inference engine generates the tokens using Qwen model. Finally the tokens are decoded back to string and finally sent as a response.

- **Fill-in-the-Middle (FIM)**

  The backend constructs prompts for FIM by:

  - Extracting the prefix (code before the cursor) and suffix (code after the cursor) from the active file.
  - Structuring the prompt clearly with:
    * **Prefix**: Code before the cursor
    * **Suffix**: Code after the cursor
    * **Middle Placeholder**: Placeholder where the AI should generate code
  - The final prompt format is:

    ```
    <|fim_prefix|> [prefix code]
    <|fim_suffix|> [suffix code]
    <|fim_middle|> [middle placeholder]
    <|end_of_text|>
    ```

- **Code Generation with Qwen Model**

  The AI model Qwen generates code based on the structured prompt, ensuring:

  - Syntactic correctness

```
def fibonacci_series(n=10):

    if n == 0 or n ==1:
        return 1

    prev1 = 1
    prev2 = 2

    for i in range(n-2):
        prev2 =<CURSOR_POSITION>


    return prev2
```

Figure 3.11: Code for which FIM is requested

```
<|fim_prefix|>def fibonacci_series(n=10):

    if n == 0 or n ==1:
        return 1

    prev1 = 1
    prev2 = 2

    for i in range(n-2):
<|fim_suffix|>


return prev2<|fim_middle|>      prev2 =
```

Figure 3.12: An example of FIM prompt structure

- Adherence to best practices
- Consistency with user coding style

3.9 User Flowchart

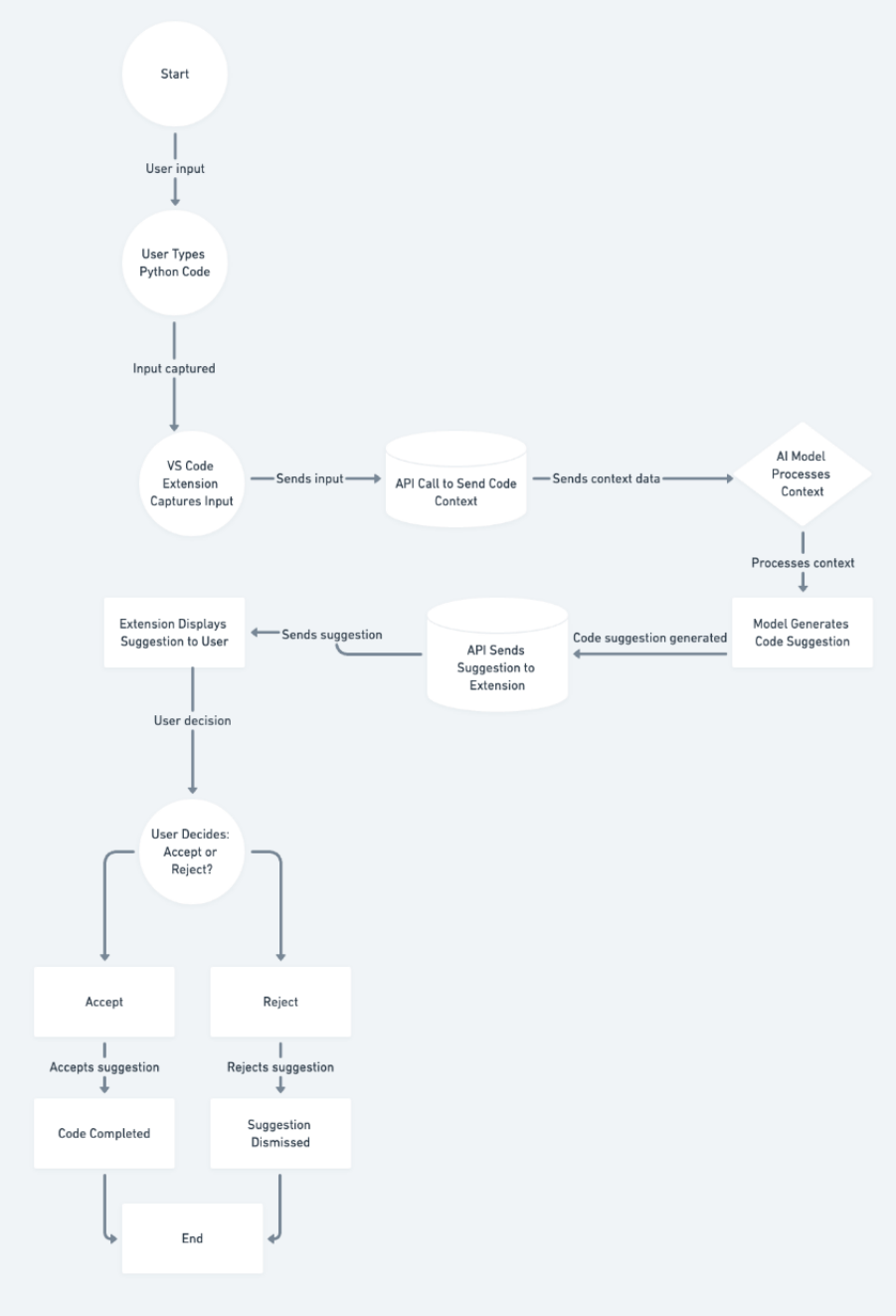


Figure 3.13: User Flowchart

4. Experimental Setup

4.1 Training

4.1.1 Libraries and GPU Used

- Python 3.12.8 with PyTorch 2.5.1 as the deep learning framework.
- CUDA 12.4 was used for GPU acceleration
- Transformers from Huggingface used for loading model and tokenizers
- Datasets from Huggingface for efficient data handling
- Tensorboard for visualizing training metric graphs
- RTX 4090 GPU for training

4.1.2 Pre-Training

Optimizer

The AdamW optimizer[17] is an improvement over Adam that decouples the weight decay from the gradient updates. Below are the formulas used:

Step 1: Compute the Moments

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \tag{4.1}$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \tag{4.2}$$

Step 2: Bias Correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \tag{4.3}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{4.4}$$

Step 3: Parameter Update with Decoupled Weight Decay

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t - \eta \cdot \lambda \cdot \theta_{t-1} \tag{4.5}$$

Where,

η is the learning rate.

β_1 is the first moment decay rate

β_2 is the second moment decay rate

ϵ is a small value to prevent division by zero.

λ is the weight decay factor.

θ_t are the model parameters at step t.

Value of parameters set in our training

η = Obtained via a learning rate scheduler

β_1= 0.9

β_2= 0.99

ϵ= 1e-8

λ= 0.01

Learning Rate Scheduler

We use a cosine learning rate scheduler with warmup and a single half-cycle. The learning rate increases linearly during the warmup phase and then decays following a half-cosine curve.

Let T be the total number of training steps, T_{warmup} the number of warmup steps, and $\eta_{\max}$ the maximum learning rate. The learning rate at step t is given by:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}}, & \text{if } t < T_{\text{warmup}} \\ \eta_{\max} \cdot \frac{1}{2}\left(1 + \cos\left(\pi \cdot \frac{t - T_{\text{warmup}}}{T - T_{\text{warmup}}}\right)\right), & \text{otherwise} \end{cases}$$

This scheduling strategy allows the model to start training stably with a low learning rate, rise up quickly, and then gradually reduce the learning rate to improve convergence during the later stages of training. A graph is provided in Figure 6.4.

Hyper Parameter Setting

Hyperparameter	**Heuristic behind the choice**
Learning Rate	Set to `9e-5`, allowing the model to adjust rapidly to the new token distribution introduced by the pruned vocabulary.
Number of Epochs	Set to 1, enabling a full pass over the dataset, suitable for cases where the base model is already pretrained.
Batch Size	A per-device batch size of **4**, combined with gradient accumulation, resulted in an effective batch size of 512.
Gradient Accumulation Steps	Accumulated gradients over **128** steps before each optimization step to simulate large batch training.
Weight Decay	Set to 0.01 to regularize the model and prevent overfitting.
Precision Format	Training used `bfloat16 (bf16)` precision to reduce memory usage and accelerate computation.
Optimizer	AdamW Optimizer used along which will had been further described in below.
Learning Rate Scheduler	A `cosine decay schedule` was used, with a **5%** warmup ratio for stable convergence.

Table 4.1: Training hyperparameters and their significance

4.1.3 Fine Tuning

Optimizer

Same AdamW optimizer with the same parameters where used for this stage.

Learning Rate Scheduler

We use a **constant learning rate scheduler with linear warmup**. In this scheduler, the learning rate increases linearly from zero to a maximum value till the warmup phase, and remains constant for the rest of the training. A constant learning rate after the warmup phase help model to properly adapt to the training data.

Let T denote the total number of training steps, T_{warmup} the number of warmup steps, and $\eta_{\max}$ the target (constant) learning rate. The learning rate η_t at a given training step t is defined as:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}}, & \text{if } t < T_{\text{warmup}} \\ \eta_{\max}, & \text{otherwise} \end{cases}$$

This type of scheduler is more suited for fine tuning purpose. The slow rise of learning rate helps model slowly adapt to the unfamiliar type of dataset then A visualization of this scheduler is shown in Figure 6.7.

Hyper Parameter Setting

Only those value are reported here which are different from the initial stage of the training

Hyperparameter	**Heuristic behind the choice**
Learning Rate	Set to `1e-4`, allowing the model to adjust rapidly to the new dataset
Number of Epochs	Set to 3, to familiarize the model with the cross-file context enabled completion
Gradient Accumulation Steps	Accumulated gradients over **32** steps before each optimization step to simulate large batch training.
Learning Rate Scheduler	A `Constant Learning Rate with warmup` was used, with a **5%** warm-up ratio for smoother adaptation.

Table 4.2: Fine Tuning hyperparameters and their heuristics

5. System Design

The system design is illustrated in Figure 5.1. It represents the working flow of the system from user keystroke to prompt generation.

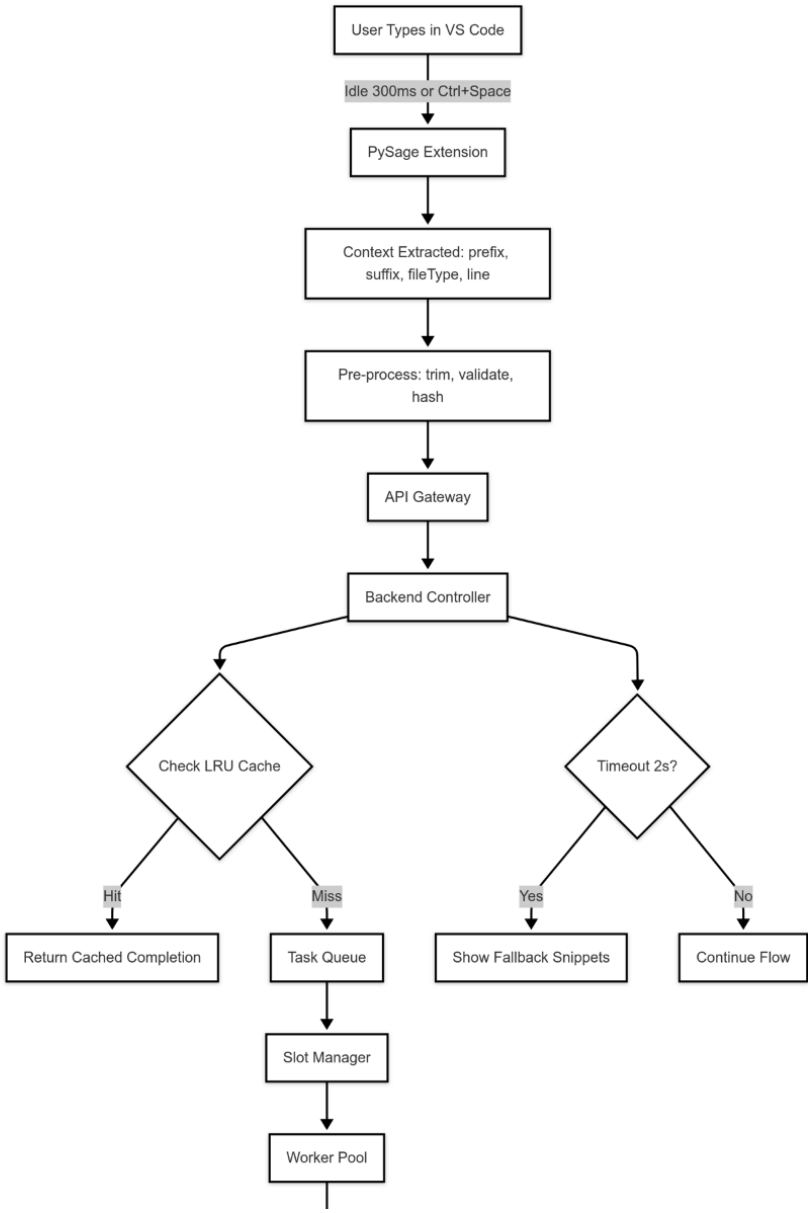


Figure 5.1: System Design

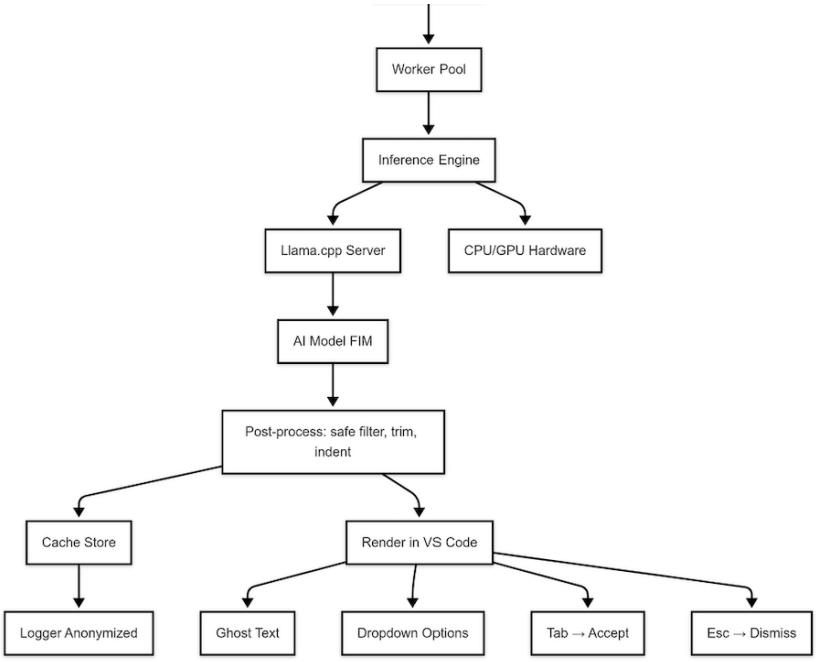


Figure 5.2: System Design - continue

5.1 Frontend System Diagram

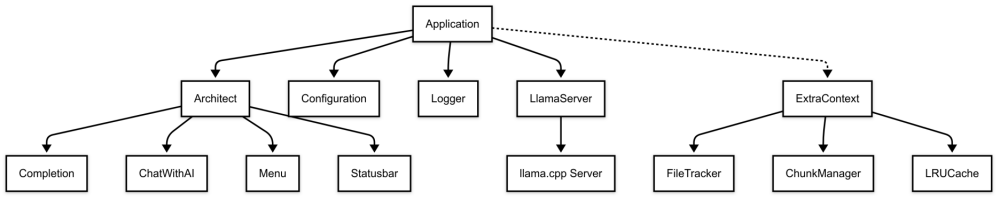


Figure 5.3: Frontend System Diagram.

The figure 5.3 displays a structured overview of what appears to be an AI-powered application's architecture, likely centered around the Llama large language model (LLM). At its core is the main Application, which integrates several key components to deliver functionality. The system includes configuration management through Architect and Configuration modules, while operational logging is handled by the Logger. A dedicated LiamaServer (possibly a misspelled LlamaServer) serves as the backend engine for LLM processing.

The application features distinct AI interaction capabilities, with Completion managing text generation tasks and ChatWithAI facilitating user conversations with the AI model. User interface elements like Menu and Statusbar provide navigation and system feedback. Underlying these features is critical infrastructure including llama.cpp (the efficient, quantized LLM backend), general Server operations, and specialized components like FileTracker for file management, ChunkManager for data segmentation (likely for handling long conver-

sations or documents), and LRUCache for performance optimization using caching strategies. The ExtraContext module suggests additional mechanisms for managing supplementary data or session-specific information. Together, this architecture appears designed to support a responsive, locally-run AI chat application with efficient resource usage and modular capabilities.

5.2 Backend System Diagram

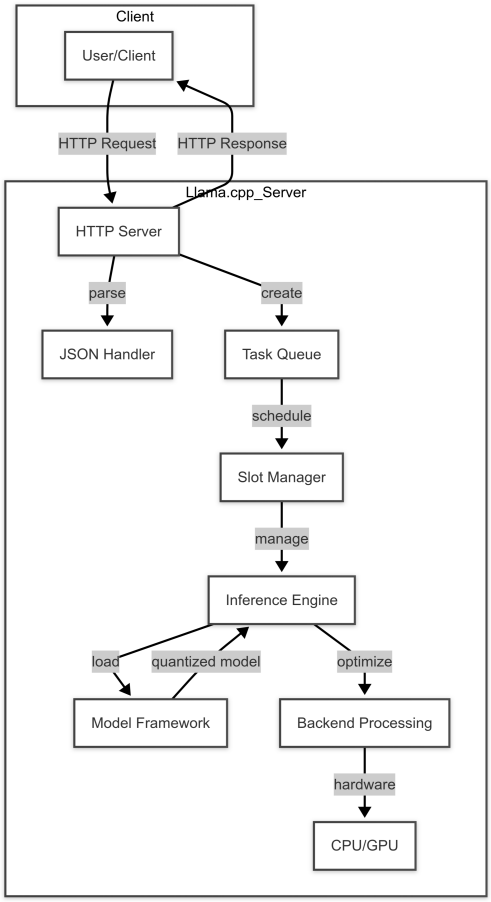


Figure 5.4: Backend System Diagram

This diagram depicts the backend workflow for a server running Llama.cpp. It starts with a client making HTTP requests, which are handled by an HTTP server. The server parses requests through a JSON Handler or creates tasks for a Task Queue, which are then scheduled by a Slot Manager. The Inference Engine manages model loading (via the Model Framework), quantized models, and backend processing, ultimately utilizing CPU/GPU hardware for inference tasks. The flow ensures efficient handling and processing of AI inference requests. We also implemented a threaded worker pool to make the backend server smoother.

5.3 Sequence Diagram

5.3.1 Extension Initialization Process

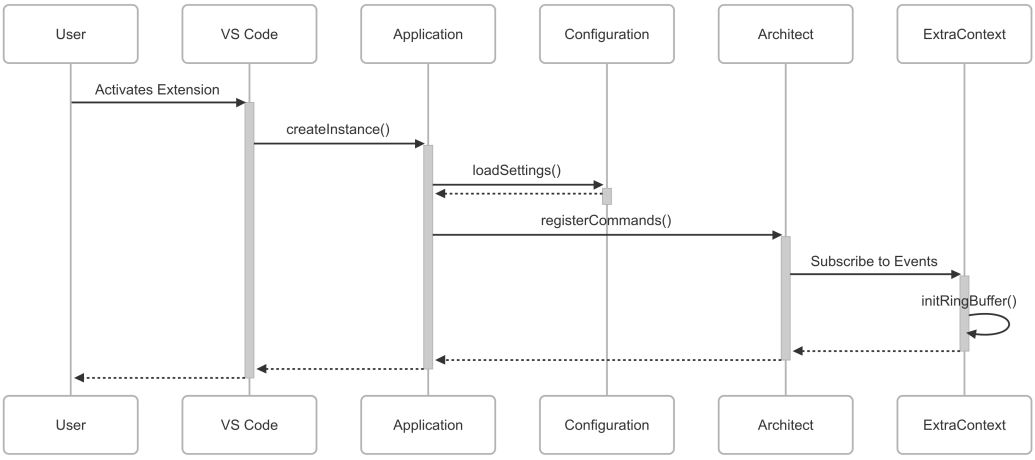


Figure 5.5: Extension Initialization Process Sequence Diagram

When PySage first loads in VS Code, it follows an orderly startup sequence. The VS Code editor creates the main Application instance, which immediately begins setting up critical services. First, it loads all user configurations like API endpoints and enabled languages from the Configuration module. Next, it initializes the Architect component to register commands and hook into editor events. Finally, it prepares the ExtraContext system, creating empty buffers to store code context. This ensures everything is ready before the user starts interacting with the extension.

5.3.2 Code Completion Workflow

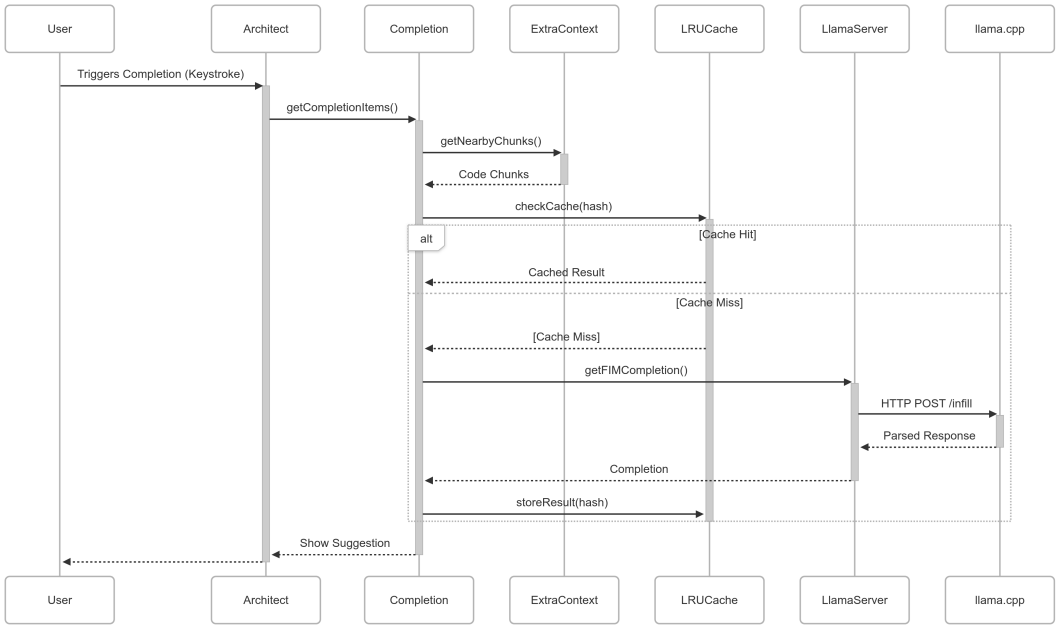


Figure 5.6: Code Completion Sequence Diagram

The completion process begins when the user triggers a suggestion. The Architect forwards this request to the Completion engine, which first gathers nearby code context from ExtraContext. It then checks if this exact code pattern has been processed before by querying the LRUCache. If found, it returns the cached suggestion immediately. For new patterns, it sends a request through LlamaServer to the external LLM, waits for the response, stores it in cache, and finally displays the suggestion. This two-path system ensures both fast responses for repeat patterns and accurate suggestions for new code.

5.3.3 Context Tracking Flow

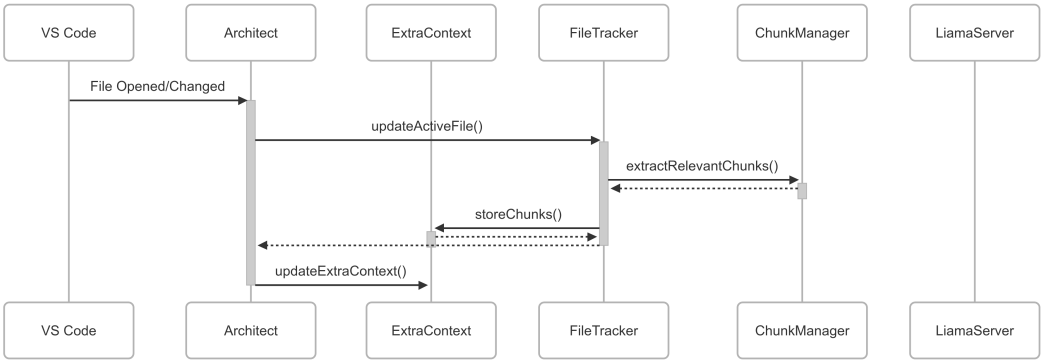


Figure 5.7: Context Tracking Sequence

As developers edit files, PySage maintains awareness of their work context. The Architect receives file change notifications from VS Code and relays them to the FileTracker. This component identifies important code segments near edits and passes them to the ChunkManager, which processes and stores them in ExtraContext's rotating memory buffer. Periodically, ExtraContext sends these updated contexts to the LLM backend, keeping its understanding of the project current. This continuous background updating is what makes PySage's suggestions contextually relevant.

5.3.4 AI Chat Workflow

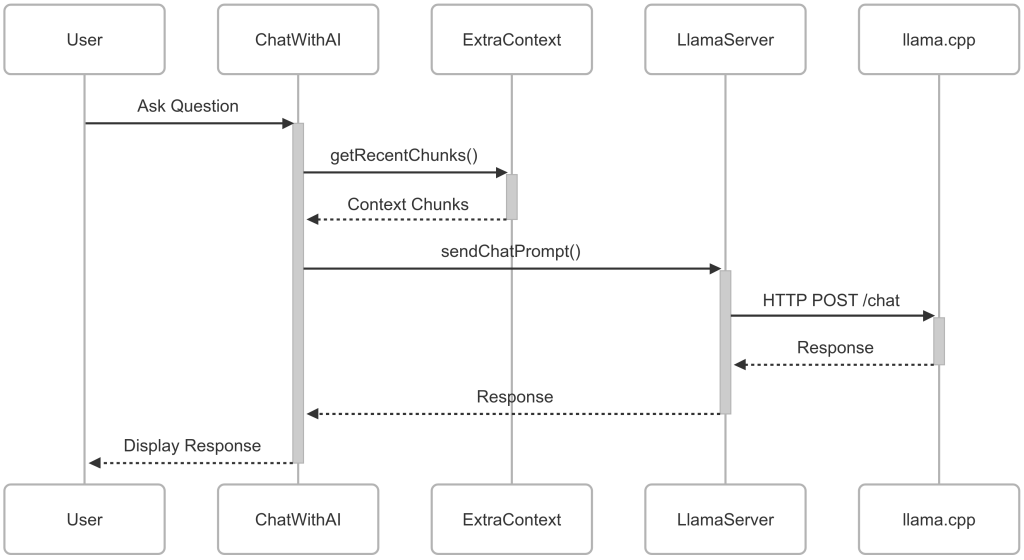


Figure 5.8: AI Chat Sequence Diagram

When using the chat feature, PySage first enriches the user's question with relevant context. The ChatWithAI component requests recent code chunks from ExtraContext, combines them with the question into a detailed prompt, and sends this to the LLM through LlamaServer. The external LLM processes this context-aware prompt and streams back its response, which PySage formats and displays in the chat interface. Unlike completions, each chat is treated as unique to provide fresh, tailored response.

5.3.5 HTTP-Connection Process

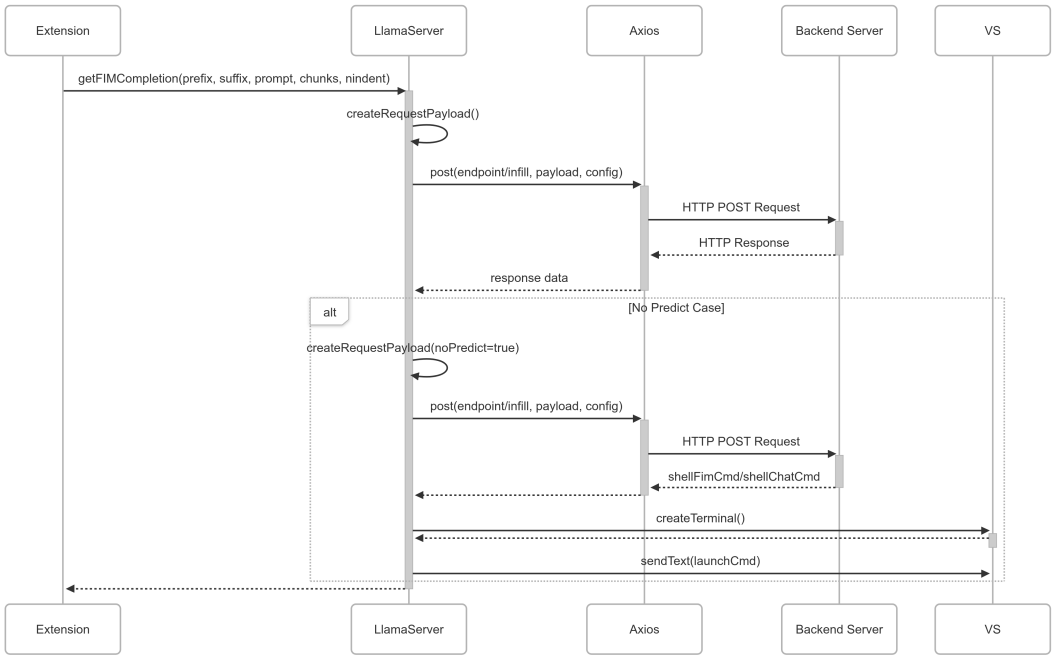


Figure 5.9: HTTP-Connection Sequence Diagram

This diagram represents the detailed steps of making a model inference request. The extension creates a request with context chunks and sends it to the backend using an HTTP POST via Axios. The backend processes the input and returns a completion. The system may update context or trigger commands based on the model's output.

LRU-Cache System Process

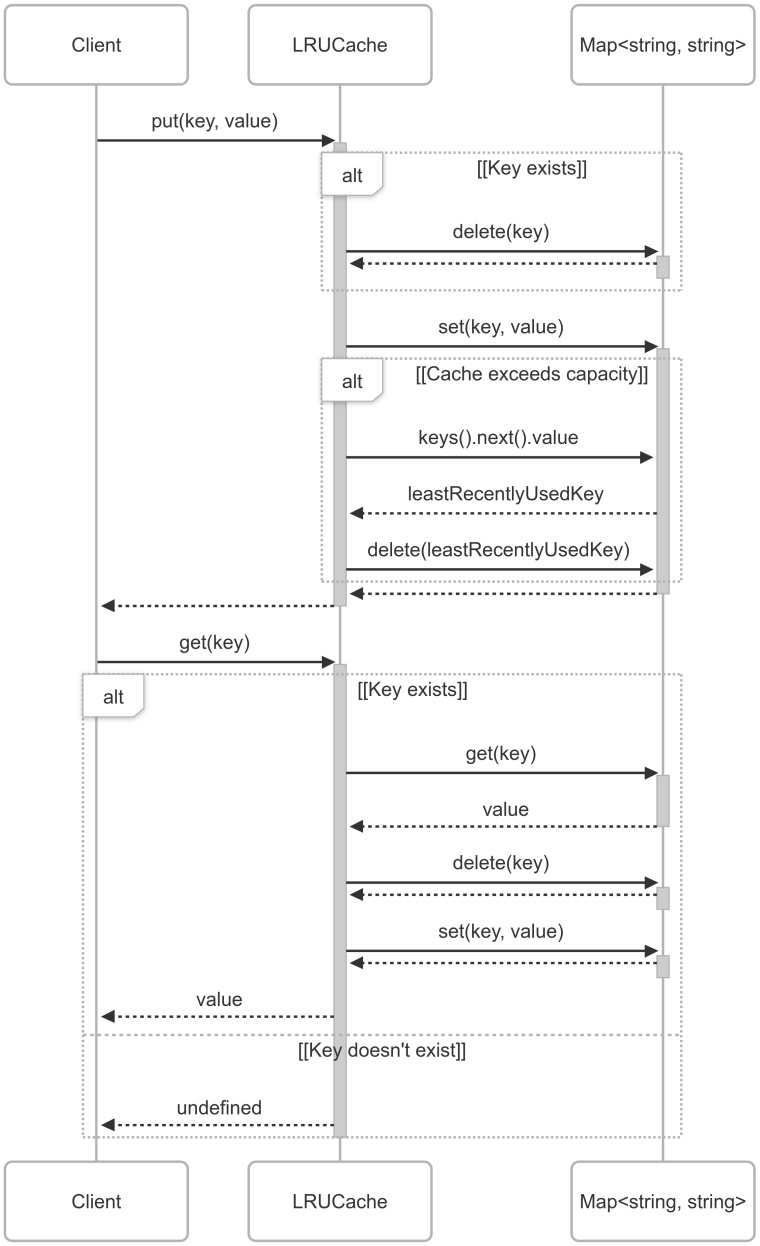


Figure 5.10: LRU Cache Sequence Diagram

This sequence explains how the Least Recently Used (LRU) cache operates. When storing a value, the system checks for existing keys or whether the cache is full. If so, it removes the least recently used item. On retrieval, it checks if the key exists and returns the value, maintaining cache consistency and access efficiency.

6. Result and Discussion

6.1 Result of vocab Pruning

To reduce model size and improve inference efficiency, we pruned the vocabulary by running the tokenizer on our dataset consisting of about a billion tokens.Those tokens that didn't appear on the datasets where removed. This led to an almost 40% reduction in embedding parameters and slightly faster decoding speeds during inference. The changes have been reported below.

Parameter	**Before**	**After**
Vocab Size	151658	90869 (↓ 60789)
Merges in tokenizer	151388	88189 (↓ 63199)
Embedding Size	135885568	54547584 (↓ 81337984)
Model Parameter	548428032	439332864 (↓ 109095168)
Non-Embed Parameter	357914240	303447296 (↓ 54466944)
Model Size (in MB)	1884	1675 (↓ 209)
Token Throughput (per sec)	7.2487	8.0966(↑ 0.8479)

Table 6.1: Result of Vocab Pruning

It is to be noted here that throughput is measured using huggingface transformers in CPU. The transformers library using pytorch which in-turn uses numpy. The numpy can posses bottleneck between Python interpreter and C api calls.

6.2 Pre-Processing Results

In the preprocessing part we spend around more than 70-80 percent of the time to curate the dataset so that our model could be feed with the fine datasets. For code structure related processing we used ruff [18] which is a fast python formatter written in rust programming language. Preprocessing includes following tasks:

- Syntax and Structural Issues: We removed files that were syntactically incorrect. This was checked using the python's native parser. Any file which didn't parsed where deemed syntactically incorrect and removed from the dataset.

- Code Containing Non-English or Irrelevant Data: Non-english comments and strings

where removed. Non-english comment where replaced with a placeholder in their place to preserve the overall code structure. Non-english comment were removed overall.

- Dependency Issues and Unresolved Imports: Using ruff, unused import and variables were removed from the code automatically.
- Code Normalization (Making Code Consistent) : To make the code consistent, all of the tabs were converted to spaces (PEP-8 Standard). Line endings were all changed to a single type (\n). White-spaces at the end of code lines were removed. Redundant line-breaks were removed (only line continuous line breaks upto 3 were kept).

```python
import json

def read_data(file_paths, get_content):
 '''
 return - JSON object
 '''
 english_comment = 'Some string to see'
 non_english_comment = "这 篇 文章 我们 在 哪儿 看 过"




 for file in file_paths: # 这个 人 我 在 哪儿 见 过 。
  with open(file, 'r') as f:
   for line in f:
    json_data = json.loads(line)
    content = get_content(json_data)
    yield content
```

Figure 6.1: One of the Original Dataset Python File

```python
import json

def read_data(file_paths, get_content):
    """
    return - JSON object
    """
    english_comment = "Some string to see"
    non_english_comment = "<<<NON ENGLISH TOKEN>>>"

    for file in file_paths:
        with open(file, 'r') as f:
            for line in f:
                json_data = json.loads(line)
                content = get_content(json_data)
                yield content
```

Figure 6.2: Normalized Dataset of above Python File

6.2.1 Training Progress and Insights

1st Stage (Further Pretraining)

Taking Qwen model, we further pretrained the model using about 4.5GB of Python code scraped between February 2024 and February 2025. The training was conducted over **1 epoch** using a batch size of **0.5M** and learning rate of **9e-5 (with cosine scheduler)**.

- We used a cosine annealing scheduler with warm-up for smoother convergence.
- Mixed-precision training was enabled to speed up training and reduce memory usage.

Loss Trends: Training and validation loss both steadily decreased throughout pretraining. Figures 6.3 show the loss curves.

Observations:

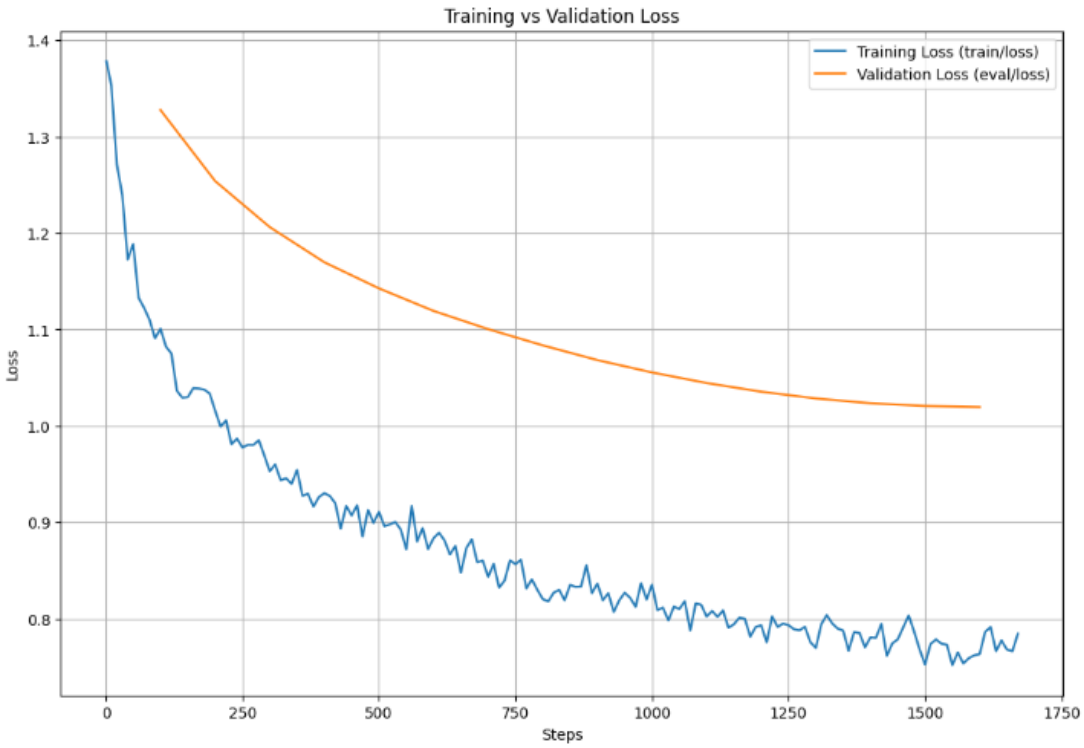


Figure 6.3: Training loss during pretraining

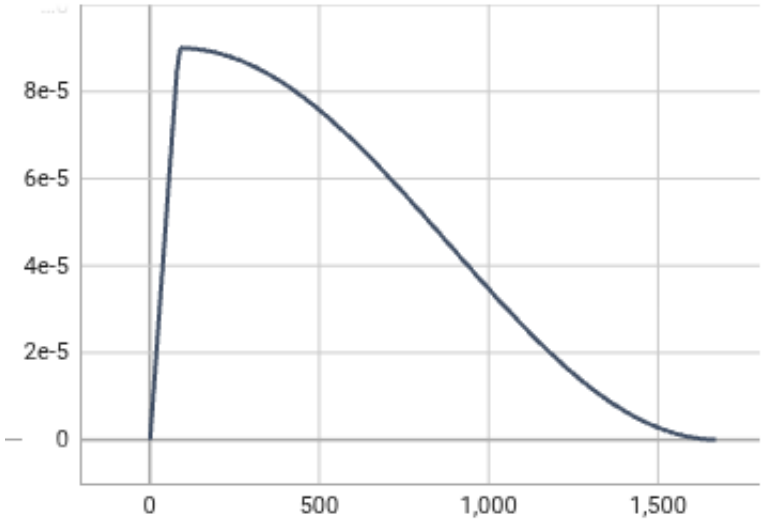


Figure 6.4: Graph of cosine scheduler with warmup

- Both training loss and validation losses were decreasing with training step.
- There still is a gap between training loss and validation loss. Since your validation set (about 5% of training data) is larger than individual training batches, it provides a more comprehensive evaluation across diverse examples. This size difference naturally creates a systematic gap in the metrics.

Evaluation on HumanEvalInfill

We evaluated the model using the HumanEvalInfill benchmark [19], focusing on its fill-in-the-middle (FIM) capabilities for Python code. The benchmark consists of prompt for different kinds of infilling tasks like single line infilling task, RandomSpan infilling and multiline infilling. All of these are designed to check a model's performance on infilling tasks. For our evaluation we used the SingleLine Infilling task where the model has to fill a missing line in the middle of a code context. It consists of about 160 samples. For each sample we generated 100 prompts. Based on the prompts pass@k was calculated. For generation sampling, we used nucleus sampling value (top_p) of 0.4 and top tokens of 30 (top_k) of 30. This setting helps produce a diverse set of output from the model.

Pass@K calculation

Pass@k is an evaluation metric commonly used for measuring the success rate of code generation models. It estimates the probability that at least one of the top-(k) generated samples passes all test cases.

$$\text{Pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \tag{6.1}$$

where,

n is the total number of generated code samples.
c is the number of correct (passing) code samples.
k is the number of samples considered for evaluation.
$\binom{n}{k}$ is the binomial coefficient, i.e., "n choose k".

Observations:

- The model's pass@1 and pass@5 scores improved compared to its pre-trained state.
- Fill-in-the-middle training enhanced the model's understanding of surrounding context, aiding its ability to infer missing logic.

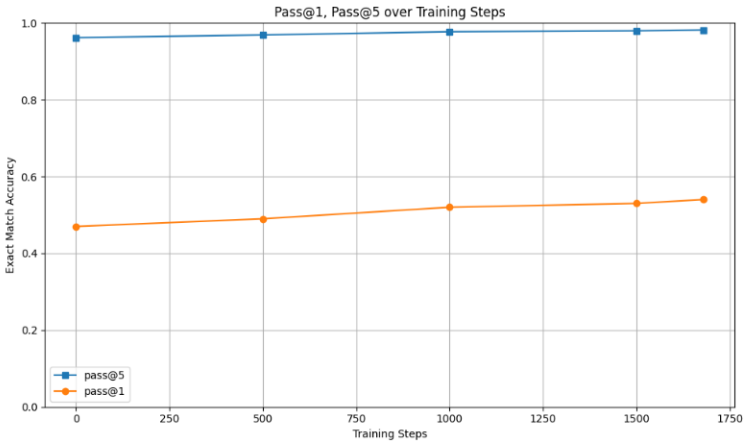


Figure 6.5: HumanEval Infilling pass@k scores

2nd Stage(Fine Tuning Phase)

The model was further fine-tuned on about 900 repositories collected between February 2025 and April 10, 2025, using context-sensitive examples designed for fill-in-the-middle and continuation tasks. The dataset consisted of about 4,500 samples in total.

- Fine-tuning was run for **3 epochs** with a smaller learning rate of **1e-4**.
- We used 90/5/5 train-validation-test split, with evaluations on the validation set after each epoch.

Loss Trends: Validation and training loss during fine-tuning are shown in Figures 6.6.

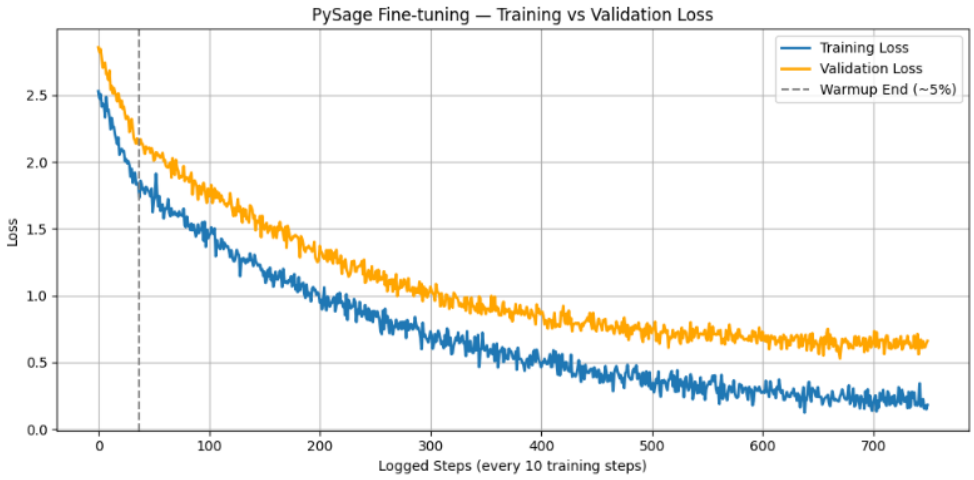


Figure 6.6: Training and validation loss during fine-tuning

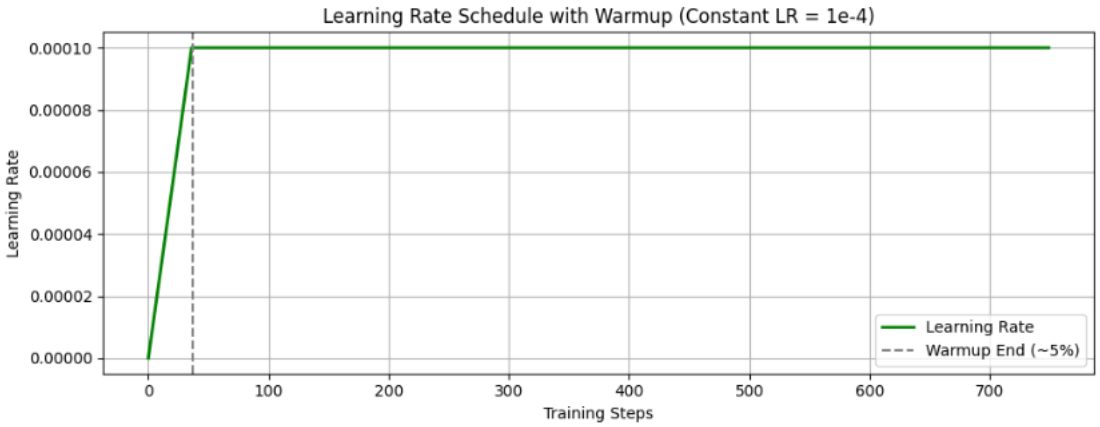


Figure 6.7: Graph of Linear scheduler with warmup

Evaluation on test data

We did an exact match (EM) on the test data. At interval of 250 we performed this test to observe how the model is performing on the test data set. EM score of the model highlight how close the model gets to the ground truth

Exact Match Mathematically it is given by the formula,

$$\text{EM} = \frac{1}{N}\sum_{i=1}^{N} 1[\hat{y}_i = y_i] \tag{6.2}$$

where,
$\hat{y}_i$ is the model's prediction token
y_i is the ground truth.
1 denotes the indicator function, which returns 1 if the prediction is an exact match and 0 otherwise.

6.3 VS Code Extension Results

The extension code is working correctly by suggesting code according to the context fetched by the extension through dependency parsing, which creates an Abstract Syntax Tree (AST). This builds the context and provides relevant code suggestions.

The figure 6.9 shows two files - 'calculate.py' and 'main.py'. In 'calculate.py', we see a complete BankAccount class definition with initialization and methods like transfer_money(), generate_report(), and generate_file(). The 'main.py' file imports this BankAccount class, and as soon as the user types 'acc = ', the IDE automatically suggests relevant code completion options that match the imported class definition. This intelligent suggestion helps the user quickly initialize the BankAccount object with the correct parameters (account holder

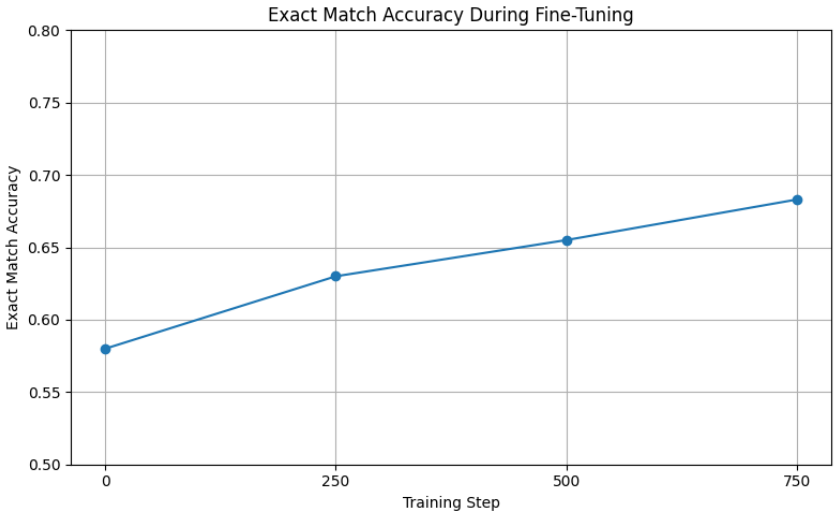


Figure 6.8: Exact Match accuracy at various steps during fine-tuning

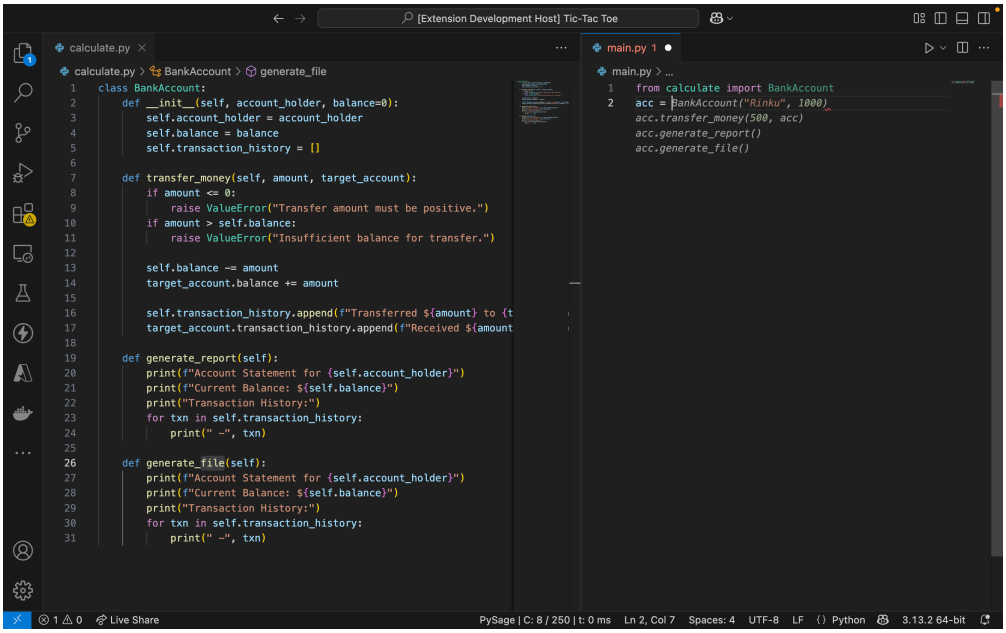


Figure 6.9: Suggestion 1

name and initial balance), demonstrating how VS Code IDE assist with code writing by understanding the context.

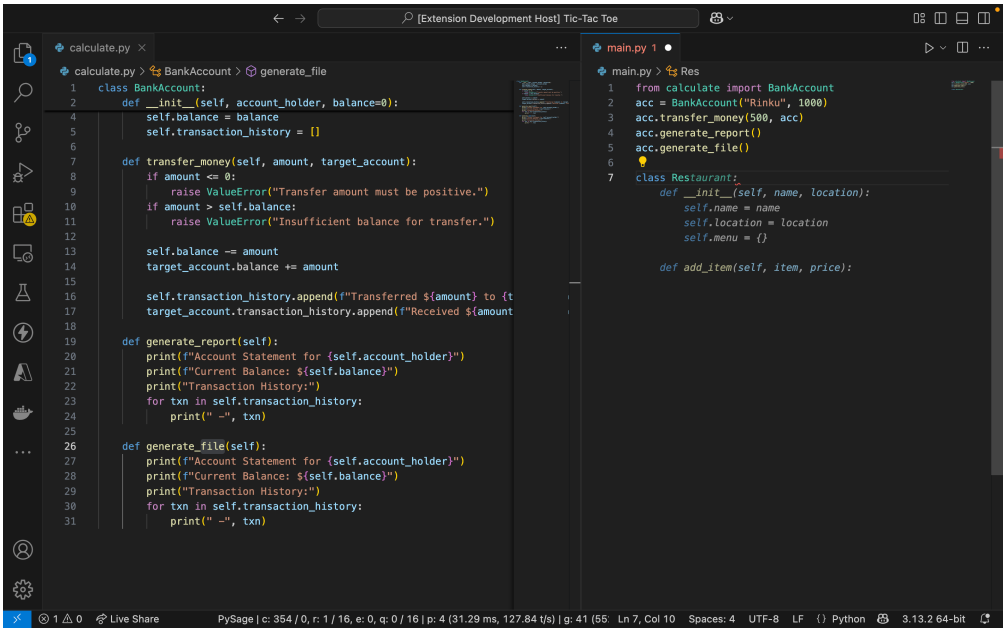


Figure 6.10: Suggestion 2

This figure 6.10 shows the same 'calculate.py' file with the BankAccount class, but now the focus has shifted to code creation in 'main.py'. As the user begins typing a new class definition (starting with 'class Rest...'), the IDE recognizes this pattern and suggests auto-completion options for creating a Restaurant class. The suggestion includes common class components like the init method with typical parameters (name and location) and basic attributes, showing how the environment helps bootstrap new class definitions based on common coding patterns.

In this final figure 6.11, while 'calculate.py' remains unchanged, we see further development in 'main.py'. As the user continues typing the restaurant-related class (now more specifically 'Restaurant_Payment'), the IDE provides more detailed suggestions. The auto-completion now includes a more complex class structure with additional attributes like 'menu' and 'orders', demonstrating how the suggestions become more sophisticated as the context becomes clearer. This illustrates the dynamic nature of our coding assistant, which adapts to the developer's evolving intent.

As shown in Figs. 6.12 and 6.13, the chat interface provides real-time programming assistance, allowing users to request specific code solutions (like palindrome checks or average calculations), debug errors, or explain concepts. Users can make direct requests such as "`give me the Python code to check whether a word is palindrome`" and receive functional implementations with clear explanations. The system handles diverse coding queries ranging

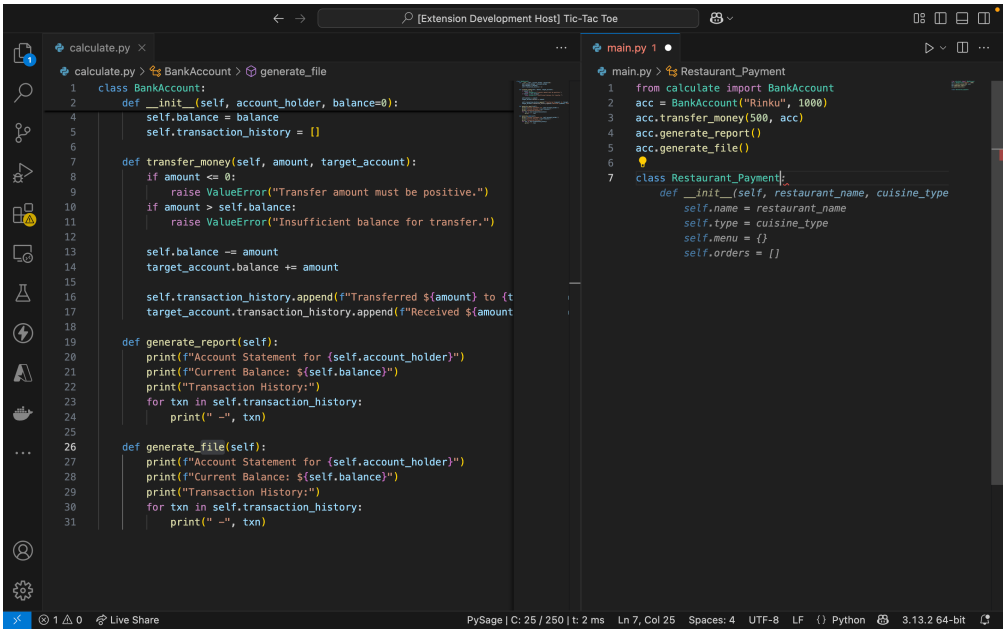


Figure 6.11: Suggestion 3

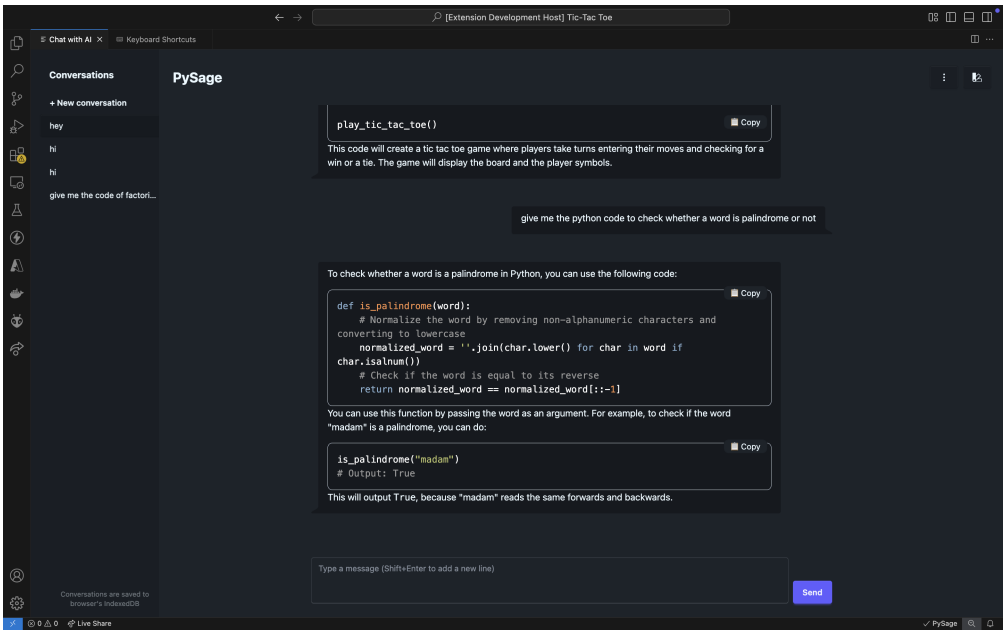


Figure 6.12: Chat with AI - 1

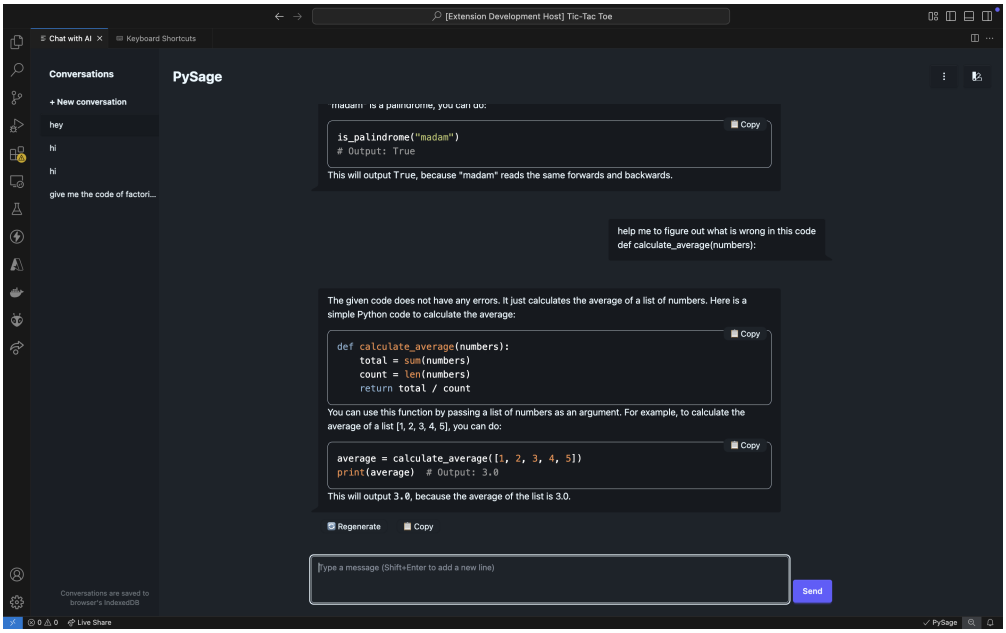


Figure 6.13: Chat with AI - 2

from syntax corrections to algorithm design across multiple languages, serving as an on-demand resource for developers at all skill levels.

Autosuggestion Workflow Summary

- Learns from existing code patterns
- Provides context-aware completions
- Adapts to code changes via file change tracking

Steps to run the Extension

- Clone the repository to download the codebase
  - Extension Server: https://github.com/ggml-org/PySage
  - Extension Client: https://github.com/khagendra7karki/PySage-Extension.git
- Setup the server
  - Using CMake build the project
  - Run the server executable with the model file path as argument
- Setup the Client

- Navigate to the root directory of the client in terminal then run the command 'npm install' to install all the required dependencies.
- Then run the command 'npm run watch' to build and run the client code.
- To activate the vs-code extension environment press F5 key in the keyboard.
- You will be redirect to the vs-code extension coding environment.Then while you make python file and start coding the extension will automatically suggest the code.

6.4 Latency Information

This figure 6.14 provides a comprehensive analysis of generation time per sample. The top section is a line plot showing individual generation times across 20 samples. The blue line with circles traces each sample's generation time, while horizontal reference lines indicate the mean (18.56 ms, red dashed) and median (19.88 ms, green dotted). As can be noticed, some points exceed the mean by a lot, these have been marked as outliers with red arrows and labels (e.g., 26.6 ms, 27.0 ms). This can be due to long prompt appearing in the request for which the system doesn't find in the cache.

The lower section in figure 6.14 is a histogram displaying the distribution of generation times. Most samples fall within the 18–26 ms range, with a few notably faster (7 ms). Vertical lines again highlight the mean and median, with text annotations indicating their values. The slight left skew of the distribution, where the median is greater than the mean, suggests the presence of a few low outliers pulling the average down. Overall, the graph reveals a relatively consistent performance with a few outlier samples that warrant further attention.

This figure 6.15 presents the latency analysis over 21 steps, breaking down the Prefill, Generate, and Total Latency times in milliseconds. Each step represents a specific point in a sequence where latency is measured.

Three distinct curves are plotted: The blue line with circular markers represents Prefill (p) latency, which remains relatively low and stable throughout, typically ranging between 25–100 ms. This indicates that the initial setup or context loading phase incurs minimal overhead across steps. The red line with square markers shows Generate (g) latency, which varies significantly across steps. At some points, such as steps 1, 2, 8, and 20, generate times spike above 500 ms, while at others like steps 7 and 11, it dips to near-zero levels. The green line with triangle markers represents the Total Latency (t), which is the combined effect of prefill and generate times. Its shape closely follows the generate curve, since generate latency is the dominant contributor to the total. The peaks of the green curve (especially at steps

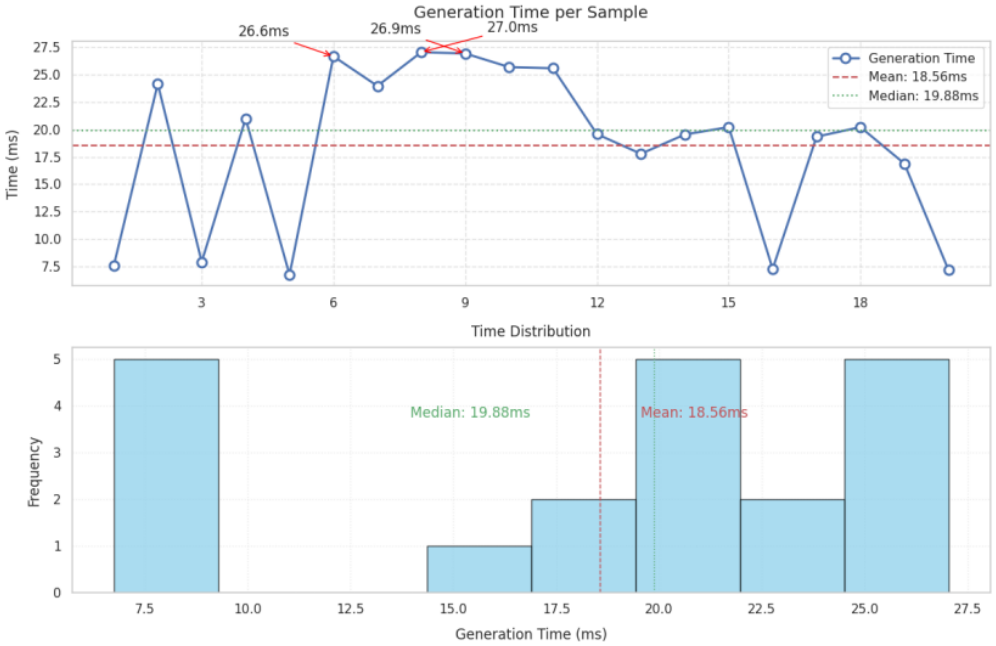


Figure 6.14: Figure showing latency information from backend

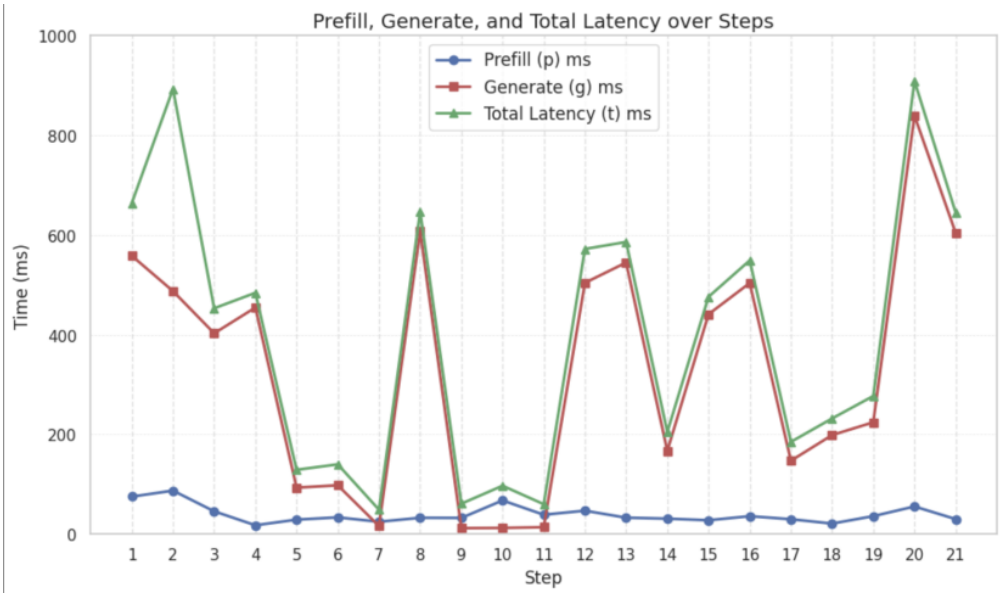


Figure 6.15: Figure showing frontend latency

2, 8, and 20) highlight the most time-consuming moments in the process.

Overall, the graph illustrates how Prefill latency stays consistent, while Generate and consequently Total latency fluctuate heavily, which suggests that generation workload varied across sampling steps.

7. Conclusion

As a part of the Bachelor of Computer Engineering (BCT) curriculum, this major project demonstrates a lightweight Python Coding Assistant designed to run efficiently on consumer-level hardware while maintaining reasonable quality and latency standards typical of modern code completion systems.

To meet our objectives, we explored various tools and techniques to optimize the use of large language models (LLMs) in resource-constrained environments. Our approach included selecting a compact pretrained model with a good performance-to-size ratio, followed by pruning the model's vocabulary to eliminate unused tokens, thereby reducing model size and improving tokenization speed.

To adapt the model to the newly pruned vocabulary and enhance its overall generative capabilities, we conducted further pretraining. This step was essential in ensuring that the model remained effective with the custom token set. We then fine-tuned the model on a domain-specific dataset tailored for Python code completion. This dataset emphasized contextual awareness, allowing the model to understand cross-file dependencies retrieved through import analysis.

These methodological steps brought us significantly closer to our initial goal. As a result, we successfully delivered a functional and responsive code completion system integrated into the Visual Studio Code environment.

While we recognize that our system does not yet rival enterprise-level code completion tools in scope or robustness, we believe this work represents a meaningful step toward democratizing LLM-powered coding assistance. It is a humble but impactful contribution toward bringing intelligent code generation into the everyday toolkit of developers.

8. Future Improvements

While the Python Coding Assistant has made significant progress, several areas can be enhanced to make it more intelligent and developer-friendly:

- **Code Error Suggestion for Definition Changes:** If a developer changes the name or signature of a function or class, the assistant should automatically detect all previous references to the old definition and warn that these will result in errors. It should also suggest appropriate updates to maintain consistency and avoid compilation or runtime issues.
- **Context-Aware Documentation Support:** Enhancing the assistant's ability to generate or update documentation based on the current context of code changes—such as renaming a function or altering its parameters—can help ensure that docstrings and comments remain accurate and up-to-date.
- **Fine-Tuning with Larger Python Datasets:** Ongoing optimization of the Qwen AI model by training it on a broader and more diverse Python coding dataset, allowing it to handle a wider variety of scenarios, libraries, and frameworks (e.g., Flask, Django, Pandas)

References

[1] Cameron R. Wolfe. Decoder-only transformers: The workhorse, 2023. Substack.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Wasi Uddin Ahmad Murali Krishna Ramanathan Ramesh Nallapati Parminder Bhatia Dan Roth Bing Xiang Yangruibo Ding, Zijian Wang. Cocomic: Code completion by jointly modeling in-file and cross-file context, 2022.

[4] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[6] Andy T. Liu, Shu-wen Li, Shih-Hsuan Yang, and Hung-yi Lee. Fused acoustic and text encoding for multimodal bilingual pretraining. *arXiv preprint arXiv:2203.07814*, 2022.

[7] Nikolas Tezak John Schulman Christine McLeavey Jerry Tworek Mark Chen Mohammad Bavarian, Heewoo Jun. Efficient training of language models to fill in the middle, 2022.

[8] Hitesh Sagtani, Rishabh Mehrotra, and Beyang Liu. Improving fim code completions via context curriculum based learning. *arXiv preprint arXiv:2412.16589*, 2024.

[9] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2016.

[10] Zixuan Wang et al. Efficient tokenization for source code models. *arXiv preprint arXiv:2205.14124*, 2022.

[11] Colin Raffel et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 2020.

[12] Srinadh Bhojanapalli Hyung Won Chung Yin-Wen Chang Chun-Sung Ferng Pu-Chin Chen, Henry Tsai. A simple and effective positional encoding for transformers. *arXiv preprint arXiv:2104.08698*, 2021.

[13] Yifei Wang Stefanie Jegelka Ali Jadbabaie Xinyi Wu1, Amir Ajorlou. On the role of attention masks and layernorm in transformers. *arXiv preprint arXiv:2405.18781*, 2024.

[14] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 6529–6541, 2019. Includes the CodeSearchNet dataset and benchmark for code retrieval tasks.

[15] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert. 2019.

[16] ggml contributors. Gguf format specification, 2023. Accessed: 2025-04-03.

[17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[18] Astral. Ruff documentation, n.d. Accessed: 2025-04-19.

[19] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.